

Rapport des Travaux Pratiques

Système Business Intelligence

Réalisé par : AMARA YASSINE

Filière : Ingénierie des systèmes d'information et Big Data (ISIBD)

Niveau : 2^{ème} Année Cycle Ingénieur

Encadré par : Pr. HRIMECH Hamid

Année universitaire : 2025 / 2026

Table des matières

Introduction	4
1 Création de la Connexion à la Base de Données	5
1.1 Configuration du Gestionnaire de Connexions OLE DB	5
2 Création du Premier Package ETL	6
2.1 Ajout d'une Tâche de Flux de Données	6
2.2 Ajout du Composant Source OLE DB	6
2.3 Configuration de la Source OLE DB	7
2.4 Ajout et Connexion du Composant Destination OLE DB	7
2.5 Configuration des Mappages de la Destination OLE DB	8
3 Modification du Package : Ajout d'une Colonne Dérivée	10
3.1 Objectif	10
3.2 Configuration de l'Expression dans l'Éditeur	10
3.3 Flux de Données après Insertion de la Colonne Dérivée	11
4 Suppression Préalable des Données : Tâche d'Exécution SQL	12
4.1 Problématique	12
4.2 Ajout de la Tâche d'Exécution SQL dans le Flux de Contrôle	12
4.3 Configuration de la Requête SQL	12
5 Intégration d'un Fichier Excel comme Source	14
5.1 Création du Gestionnaire de Connexions Excel	14
5.2 Ajout du Flux Source Excel vers Destination	14
6 Intégration d'un Fichier CSV comme Source	16
6.1 Création du Gestionnaire de Connexions Fichier Plat	16
6.2 Ajout du Flux Fichier Plat vers Destination	16
7 Exécution Finale du Package	17
7.1 Résultat de l'Exécution	17
8 TP 2 : Création du Package et des Gestionnaires de Connexions	18
8.1 Nouveau Package SSIS : Import TXT vers SQL	18
8.2 Gestionnaires de Connexions	18
9 TP 2 : Configuration du Flux de Données	19
9.1 Source du Fichier Plat vers Destination OLE DB	19
9.2 Configuration de la Destination OLE DB	19

9.3	Mappages des Colonnes	20
9.4	Exécution du Flux de Données Simple	20
10	TP 2 : Personnalisation — Tâche SQL et Script Task	22
10.1	Ajout d'une Tâche d'Exécution SQL	22
10.2	Ajout d'une Tâche de Script (Script Task)	22
11	TP 2 : Colonne Dérivée dans le Flux de Données	24
11.1	Insertion d'un Composant Colonne Dérivée	24
12	TP 2 : Chargement de Plusieurs Fichiers Plats — Conteneur Foreach	25
12.1	Objectif	25
12.2	Configuration du Conteneur Foreach	25
12.3	Flux de Données à l'Intérieur de la Boucle	26
12.4	Exécution Réussie de la Boucle	26
13	TP 2 : Extraction vers un Fichier Plat et Flux Multiples	27
13.1	Source OLE DB vers Destination Fichier Plat	27
14	TP 2 : Gestion des Erreurs avec Event Handlers	28
14.1	Configuration du Gestionnaire d'Événements OnError	28
14.2	Configuration du Script de Gestion d'Erreur	28
14.3	Types d'Erreurs Gérés	29
15	TP 3 : Introduction et Présentation du Projet BikeStores	30
15.1	Contexte et Objectifs	30
15.2	Structure de la Base de Données Opérationnelle	30
15.3	Modélisation du Data Warehouse BikeStoresDW	30
16	TP 3 : Nettoyage et Initialisation des Tables	32
16.1	Problématique de la Réexécution	32
16.2	Tâche d'Exécution SQL : Fact Vente	32
17	TP 3 : Alimentation de la Dimension Store	34
17.1	Vue d'Ensemble du Flux de Contrôle Initial	34
17.2	Alimentation de la Table État	34
17.3	Alimentation de la Table Ville	35
17.4	Alimentation de la Table Store	35
18	TP 3 : Alimentation de la Dimension Produit	37
18.1	Vue d'Ensemble du Flux de Contrôle Étendu	37
18.2	Alimentation des Tables Marque et Catégorie	37
18.3	Alimentation de la Table Produit	38
19	TP 3 : Alimentation de la Table des Faits Ventes	40
19.1	Flux de Contrôle Final	40
19.2	Configuration du Flux de Données Ventes	41
20	TP 3 : Exécution Finale du Package BikeStoresDW	43
20.1	Résultat de l'Exécution Complète	43

Partie II : Visualisation avec Microsoft Power BI	46
21 TP Power BI 1 : Tableau de Bord Superstore	47
21.1 Présentation du Projet	47
21.2 Page 1 — Vue Globale des Ventes	47
21.3 Page 2 — Analyse Détaillée par Région et Catégorie	48
22 TP Power BI 2 : Tableau de Bord import-export	50
22.1 Présentation du Projet	50
22.2 Tableau de Bord import-export	50
23 TP Power BI 3 : Tableau de Bord Stocks Restauration	52
23.1 Présentation du Projet	52
23.2 Tableau de Bord Stocks	52
24 TP Power BI 4 : Tableau de Bord Retail Australie	54
24.1 Présentation du Projet	54
24.2 Modèle de Données en Étoile	54
24.3 Page 1 — Vue Globale des Ventes par Chaîne et État	55
24.4 Page 2 — Analyse par Manager et Suburb	56
Conclusion	57

Introduction

Ce rapport présente les travaux pratiques réalisés dans le cadre du module **Système Business Intelligence**. Ces TP portent sur la prise en main de **SQL Server Integration Services (SSIS)**, outil ETL de la suite Microsoft BI, permettant d'extraire, transformer et charger des données depuis des sources hétérogènes (base SQL Server, fichiers Excel, fichiers CSV) vers une base de données de destination, ainsi que sur l'outil de visualisation **Microsoft Power BI**.

Chapitre 1 Création de la Connexion à la Base de Données

1.1 Configuration du Gestionnaire de Connexions OLE DB

La première étape consiste à créer un gestionnaire de connexions permettant à SSIS de communiquer avec la base de données SQL Server. Pour cela, un clic droit dans la zone *Gestionnaires de connexions* en bas de l'interface ouvre le menu de création.

On sélectionne **Nouvelle connexion OLE DB**, ce qui ouvre la fenêtre *Gestionnaire de connexions*. Les paramètres suivants ont été renseignés :

- **Fournisseur** : OLE DB natif / SQL Server Native Client 11.0
- **Nom du serveur** : DESKTOP-RNQCLFN\SQLEXPRESS
- **Authentification** : Windows Authentication
- **Base de données** : gestion_livres

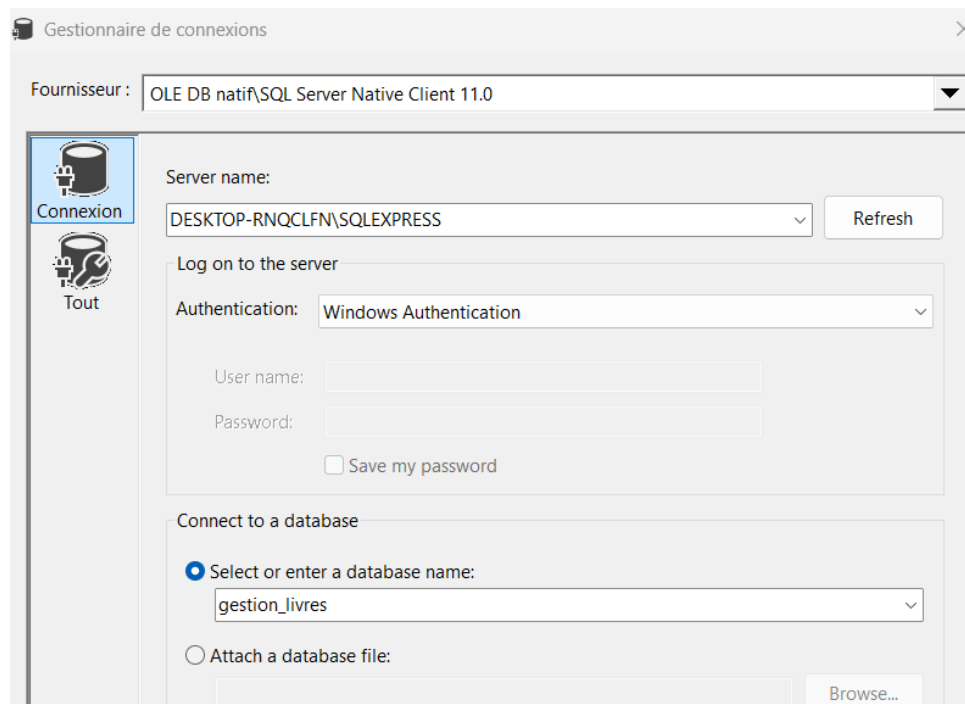


FIGURE 1.1 – Configuration du gestionnaire de connexions OLE DB vers `gestion_livres`

Après validation, la connexion apparaît dans la barre des gestionnaires de connexions et sera réutilisée dans tous les composants source et destination du package.

Chapitre 2 Création du Premier Package ETL

2.1 Ajout d'une Tâche de Flux de Données

Dans l'onglet **Flux de contrôle**, on fait glisser une **Tâche de flux de données** depuis la boîte à outils vers la zone de conception. Cette tâche constitue le conteneur principal dans lequel seront définis la source, les transformations et la destination des données.

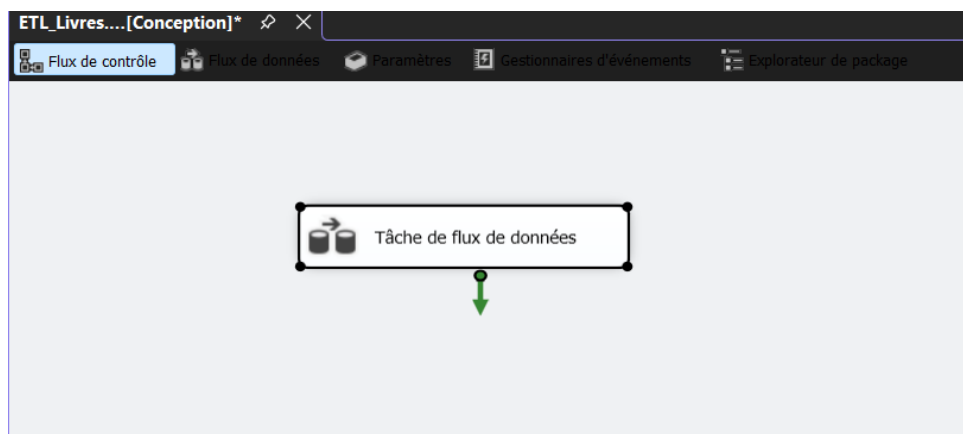


FIGURE 2.1 – Tâche de flux de données ajoutée dans le flux de contrôle

2.2 Ajout du Composant Source OLE DB

En double-cliquant sur la tâche de flux de données, on bascule vers l'onglet **Flux de données**. On y ajoute un composant **Source OLE DB** depuis la boîte à outils en le faisant glisser dans la zone de conception.



FIGURE 2.2 – Composant Source OLE DB ajouté dans le flux de données

2.3 Configuration de la Source OLE DB

Un double-clic sur le composant Source OLE DB ouvre son éditeur. On y configure le gestionnaire de connexions, le mode d'accès aux données ainsi que la table source :

- **Gestionnaire de connexions** : DESKTOP-RNQCLFN\SQLEXPRESS.gestion_livres
- **Mode d'accès aux données** : Table ou vue
- **Nom de la table ou de la vue** : [dbo].[Auteur]

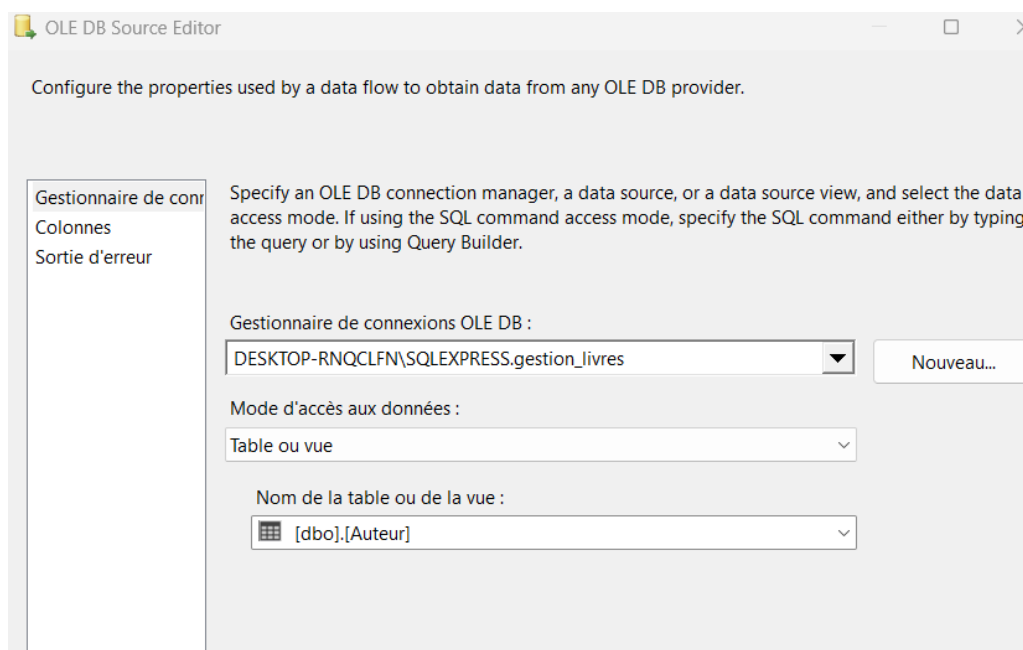


FIGURE 2.3 – Éditeur de la Source OLE DB — sélection de la table [dbo] . [Auteur]

2.4 Ajout et Connexion du Composant Destination OLE DB

Après la source, on ajoute un composant **Destination OLE DB** dans la zone de flux de données. Pour relier la Source à la Destination, on clique sur la Source OLE DB et on fait

glisser la flèche bleue (flux de données) vers la Destination. La flèche rouge sur la destination indique qu'elle n'est pas encore configurée.

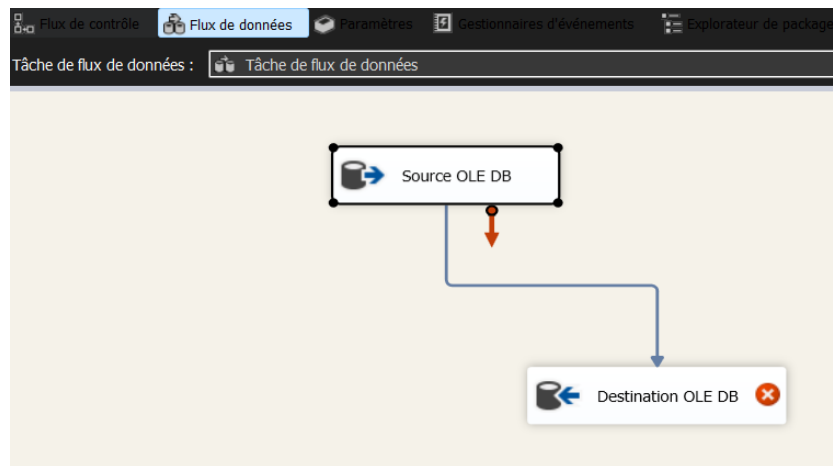


FIGURE 2.4 – Connexion entre Source OLE DB et Destination OLE DB — flèche bleue de flux de données

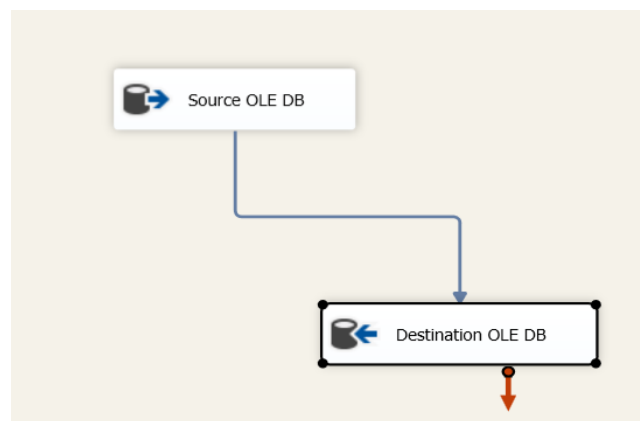


FIGURE 2.5 – Vue du flux de données avec la liaison entre source et destination établie

2.5 Configuration des Mappages de la Destination OLE DB

Dans l'éditeur de la Destination OLE DB, l'onglet **Mappages** affiche la correspondance automatique entre les colonnes d'entrée (issues de la source) et les colonnes de la table de destination. Les colonnes `NUMERO_A`, `NOM` et `Type_Auteur` sont mappées vers leurs équivalents dans la table cible.

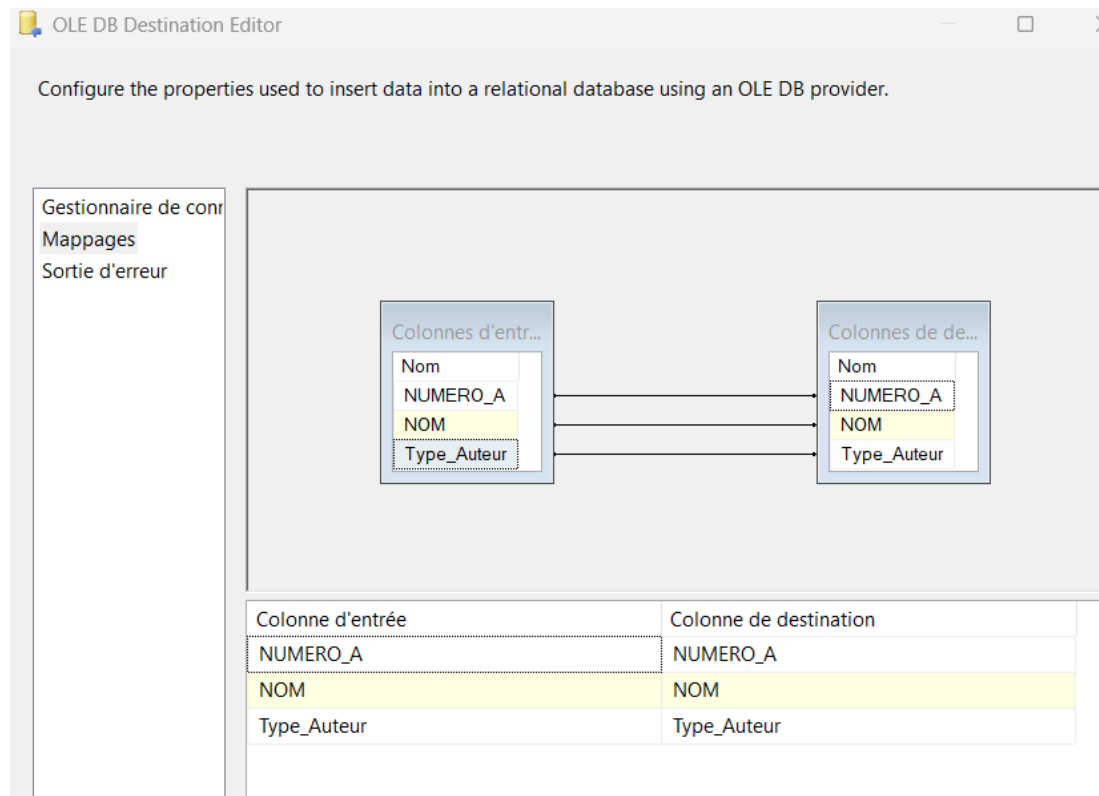


FIGURE 2.6 – Mappages des colonnes dans l'éditeur de la Destination OLE DB

Chapitre 3 Modification du Package : Ajout d'une Colonne Dérivée

3.1 Objectif

L'objectif est d'enrichir le flux de données en ajoutant une colonne calculée **Type_Auteur** dont la valeur dépend du numéro d'auteur. La règle appliquée est la suivante :

```
NUMERO_A >= 100000 ? "nouveau auteur" : "ancien auteur"
```

Si **NUMERO_A** est supérieur ou égal à 100 000, la valeur sera *"nouveau auteur"*, sinon *"ancien auteur"*.

3.2 Configuration de l'Expression dans l'Éditeur

Dans le flux de données, on insère un composant **Colonne dérivée** entre la Source OLE DB et la Destination OLE DB. Dans son éditeur (*Derived Column Transformation Editor*), on définit :

- **Nom de la colonne dérivée** : **Type_Auteur**
- **Action** : Ajouter comme nouvelle colonne
- **Expression** : **NUMERO_A >= 100000 ? "nouveau auteur" : "ancien auteur"**
- **Type de données** : chaîne Unicode [DT_WSTR]

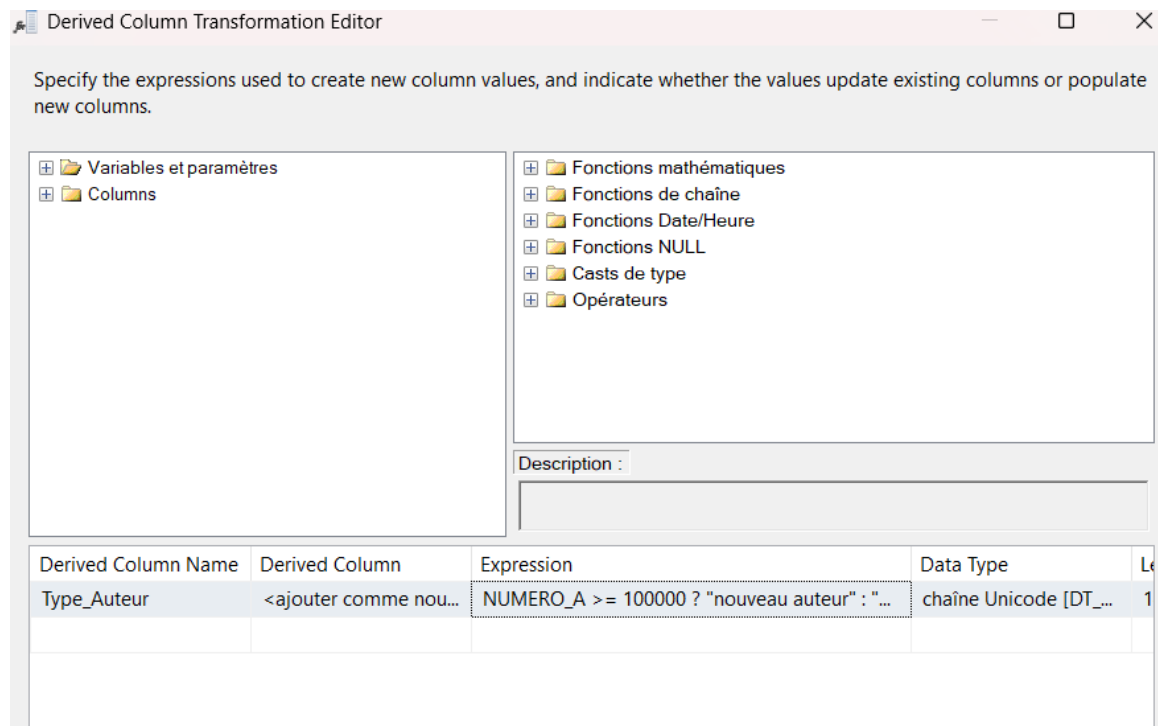


FIGURE 3.1 – Éditeur Colonne dérivée — expression conditionnelle pour Type_Auteur

3.3 Flux de Données après Insertion de la Colonne Dérivée

Après insertion et configuration, le composant **Colonne dérivée** s'intercale entre la Source et la Destination. La flèche orange indique que la Destination doit être reconfigurée pour prendre en compte la nouvelle colonne.

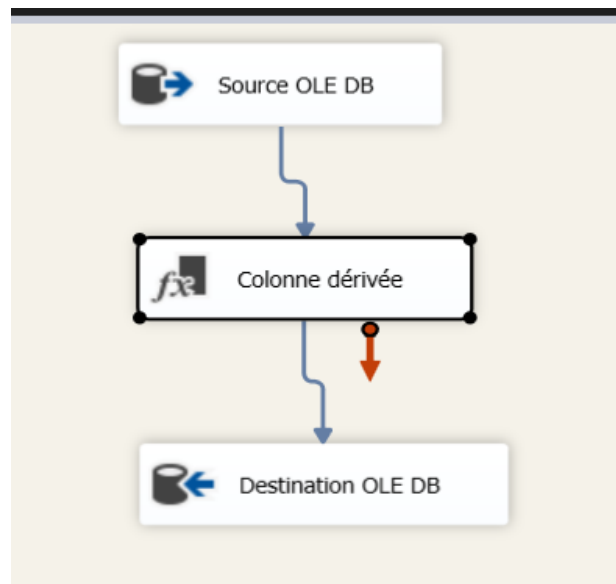


FIGURE 3.2 – Flux : Source OLE DB → Colonne dérivée → Destination OLE DB

Chapitre 4 Suppression Préalable des Données : Tâche d'Exécution SQL

4.1 Problématique

À chaque exécution du package, les données s'ajoutent à la table de destination, provoquant des doublons. Pour garantir un chargement propre à chaque exécution, on ajoute une étape de suppression préalable des données de la table destination.

4.2 Ajout de la Tâche d'Exécution SQL dans le Flux de Contrôle

De retour dans l'onglet **Flux de contrôle**, on ajoute une **Tâche d'exécution de requêtes SQL** au-dessus de la Tâche de flux de données. On relie ensuite les deux tâches par une flèche verte (contrainte de précédence : la tâche de flux de données ne s'exécute que si la tâche SQL réussit).

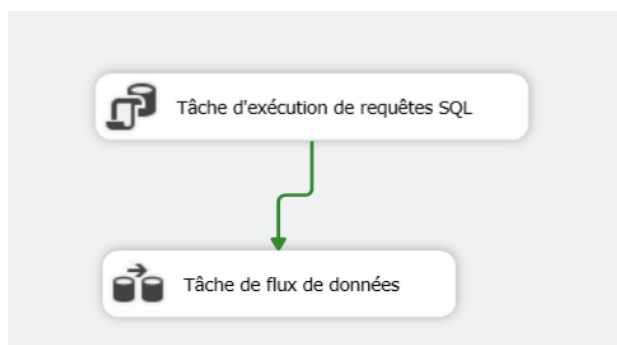


FIGURE 4.1 – Flux de contrôle : Tâche d'exécution SQL précédant la Tâche de flux de données

4.3 Configuration de la Requête SQL

Dans les propriétés de la tâche SQL, on renseigne les paramètres suivants :

- **ConnectionType** : OLE DB
- **Connection** : DESKTOP-RNQCLFN\SQLEXPRESS.gestion_livres
- **SQLSourceType** : Entrée directe
- **SQLStatement** : DELETE FROM Auteur_Copy;
- **ResultSet** : Aucun

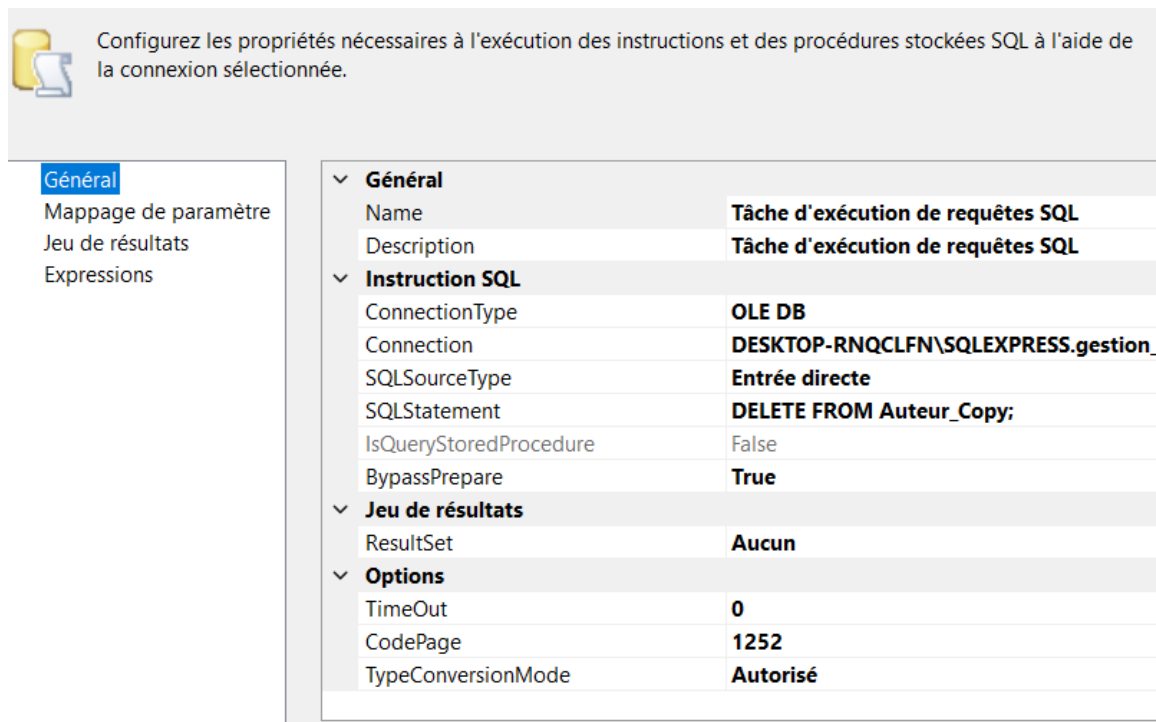


FIGURE 4.2 – Propriétés de la Tâche d'exécution SQL — instruction DELETE FROM Auteur_Copy

Chapitre 5 Intégration d'un Fichier Excel comme Source

5.1 Création du Gestionnaire de Connexions Excel

Pour intégrer une source Excel, on crée un nouveau gestionnaire de connexions de type **EXCEL** via un clic droit dans la zone des gestionnaires de connexions. La fenêtre de configuration demande le chemin du fichier, la version Excel et indique si la première ligne contient les noms de colonnes.

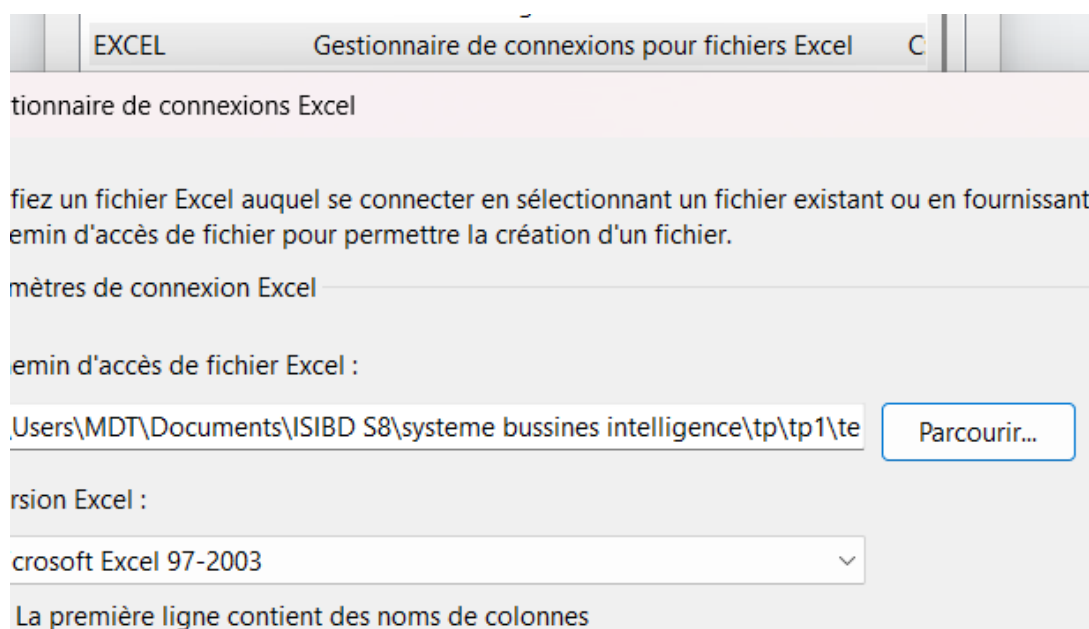


FIGURE 5.1 – Configuration du gestionnaire de connexions Excel

5.2 Ajout du Flux Source Excel vers Destination

Dans le flux de données, on ajoute un second flux indépendant composé d'une **Source Excel** reliée à une **Destination OLE DB 1**. Ce flux fonctionne en parallèle du flux SQL Server déjà existant.

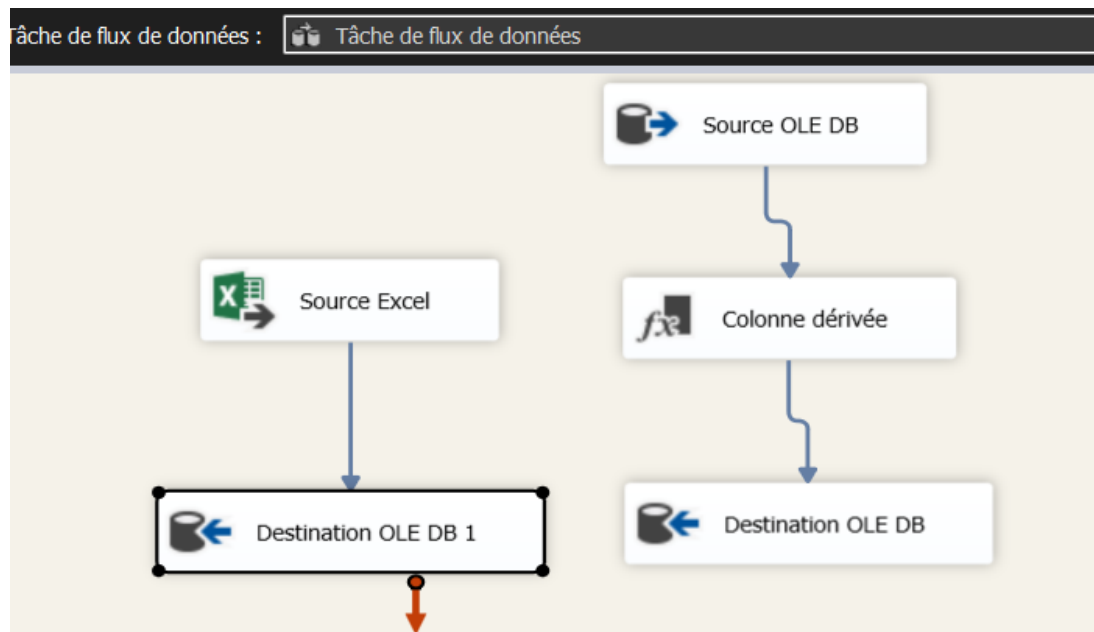


FIGURE 5.2 – Flux de données avec le flux Excel en parallèle du flux SQL Server

Chapitre 6 Intégration d'un Fichier CSV comme Source

6.1 Création du Gestionnaire de Connexions Fichier Plat

Pour lire un fichier CSV, on crée un gestionnaire de connexions de type **FLATFILE**. Les paramètres configurés sont le chemin du fichier, le format délimité, la page de code et le délimiteur de ligne.

6.2 Ajout du Flux Fichier Plat vers Destination

Un troisième flux indépendant est ajouté dans la tâche de flux de données, composé d'une **Source du fichier plat** reliée à une **Destination OLE DB 2**. Les trois flux de données fonctionnent désormais en parallèle.

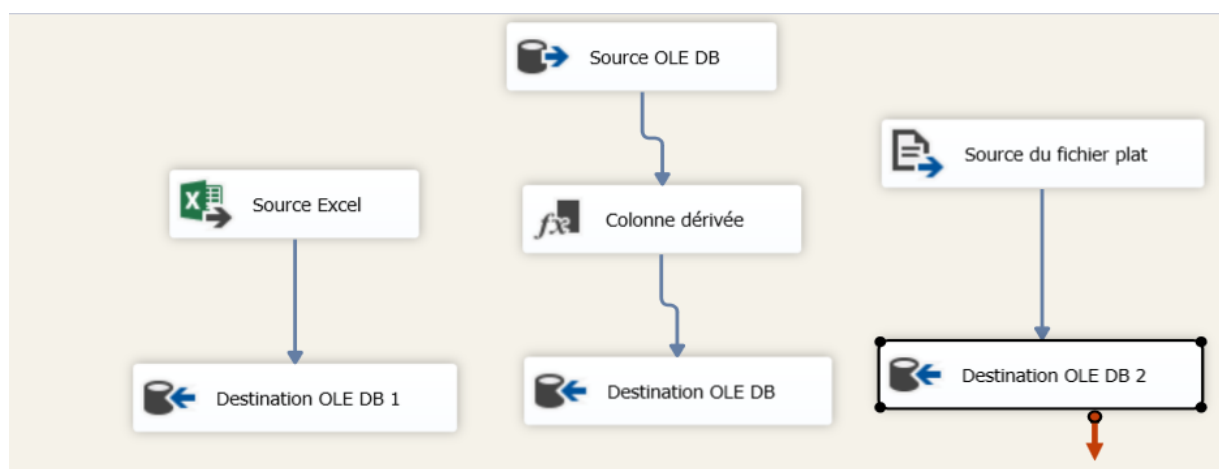


FIGURE 6.1 – Flux de données complet : trois flux en parallèle (SQL Server, Excel, fichier CSV)

Chapitre 7 Exécution Finale du Package

7.1 Résultat de l'Exécution

Après avoir configuré l'ensemble des composants, le package a été exécuté via le menu *Débogage* ou le bouton *Démarrer*. Tous les composants affichent une **icône verte** confirmant le succès de chaque étape :

- **Source OLE DB** : lecture réussie depuis [dbo].[Auteur]
- **Colonne dérivée** : calcul de `Type_Auteur` effectué correctement
- **Destination OLE DB** : chargement dans la table destination réussi
- **Source Excel** : lecture du fichier Excel réussie
- **Destination OLE DB 1** : chargement des données Excel réussi
- **Source du fichier plat** : lecture du fichier CSV réussie
- **Destination OLE DB 2** : chargement des données CSV réussi

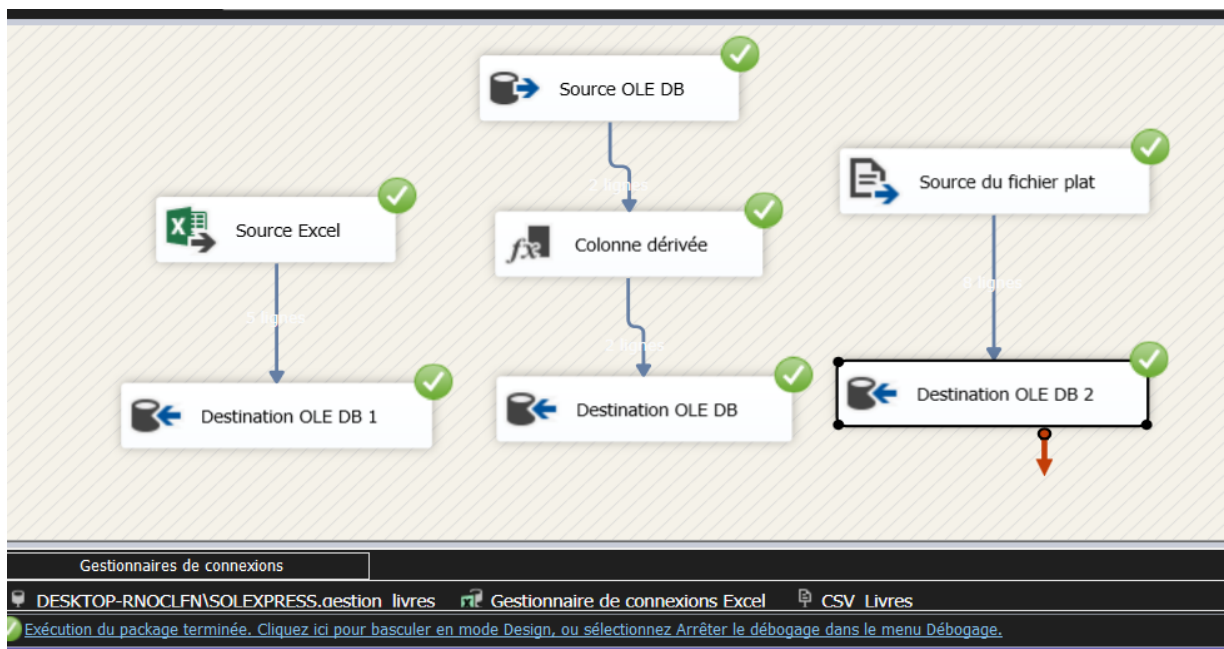


FIGURE 7.1 – Exécution réussie du package — tous les composants affichent une icône verte

Le message « *Exécution du package terminée* » visible en bas de l'interface confirme que le pipeline ETL complet s'est exécuté sans erreur. Les trois gestionnaires de connexions actifs sont visibles : `DESKTOP-RNOCLFN\SOLEXPRESS.gestion_livres`, `Gestionnaire de connexions Excel` et `CSV Livres`.

Chapitre 8 TP 2 : Création du Package et des Gestionnaires de Connexions

8.1 Nouveau Package SSIS : Import TXT vers SQL

Un nouveau package SSIS est créé et nommé **Import TXT vers SQL**. Dans l'onglet **Flux de contrôle**, une **Tâche de flux de données** portant ce nom est ajoutée. Une flèche verte de contrainte de précedence est visible en bas de la tâche, indiquant qu'elle sera liée à d'autres tâches ultérieurement.

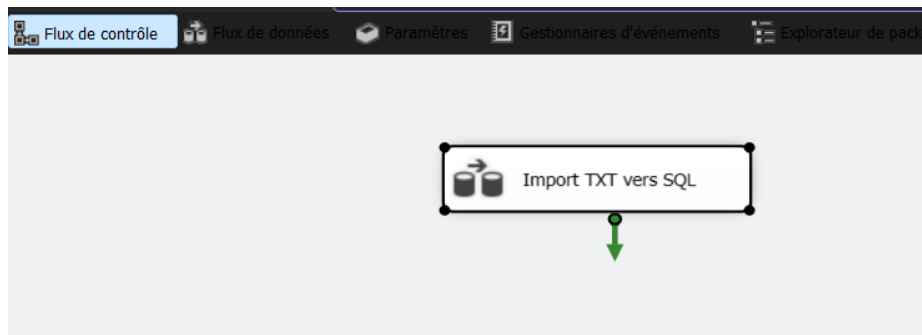


FIGURE 8.1 – Flux de contrôle : tâche de flux de données *Import TXT vers SQL*

8.2 Gestionnaires de Connexions

Deux gestionnaires de connexions sont créés pour ce package :

- **DESKTOP-RNQCLFN\SQLEXPRESS.TP_ETL** : connexion OLE DB vers la base de données SQL Server TP_ETL.
- **FlatFile_TestETL** : connexion de type *Fichier plat* pointant vers le fichier texte source.

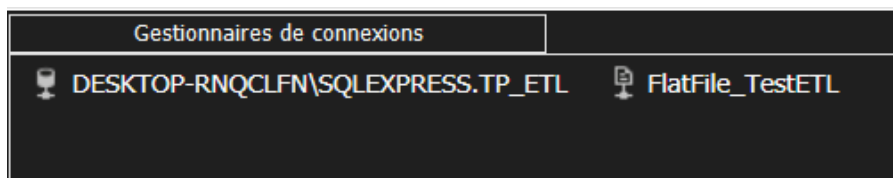


FIGURE 8.2 – Gestionnaires de connexions : OLE DB (SQL Server) et Fichier plat

Chapitre 9 TP 2 : Configuration du Flux de Données

9.1 Source du Fichier Plat vers Destination OLE DB

Dans l'onglet **Flux de données** de la tâche *Import TXT vers SQL*, on ajoute un composant **Source du fichier plat** relié par une flèche bleue à un composant **Destination OLE DB**. La flèche orange sur la destination indique que celle-ci n'est pas encore complètement configurée (mappages en attente).

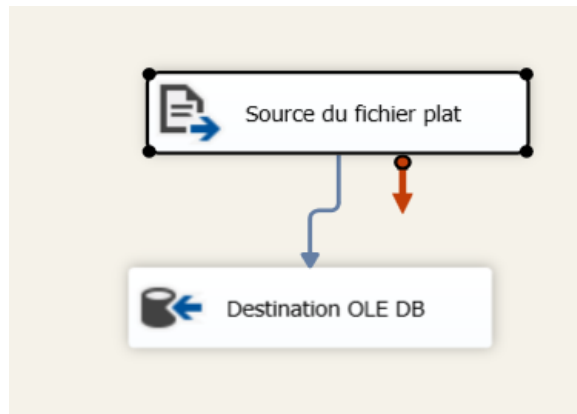


FIGURE 9.1 – Flux de données : Source du fichier plat → Destination OLE DB (configuration en cours)

9.2 Configuration de la Destination OLE DB

L'éditeur de la **Destination OLE DB** est configuré avec les paramètres suivants :

- **Gestionnaire de connexions OLE DB** : DESKTOP-RNQCLFN\SQLEXPRESS.TP_ETL
- **Mode d'accès aux données** : Table ou vue — chargement rapide
- **Nom de la table ou de la vue** : [Destination OLE DB]
- Options activées : *Verrou de table* et *Vérifier les contraintes*

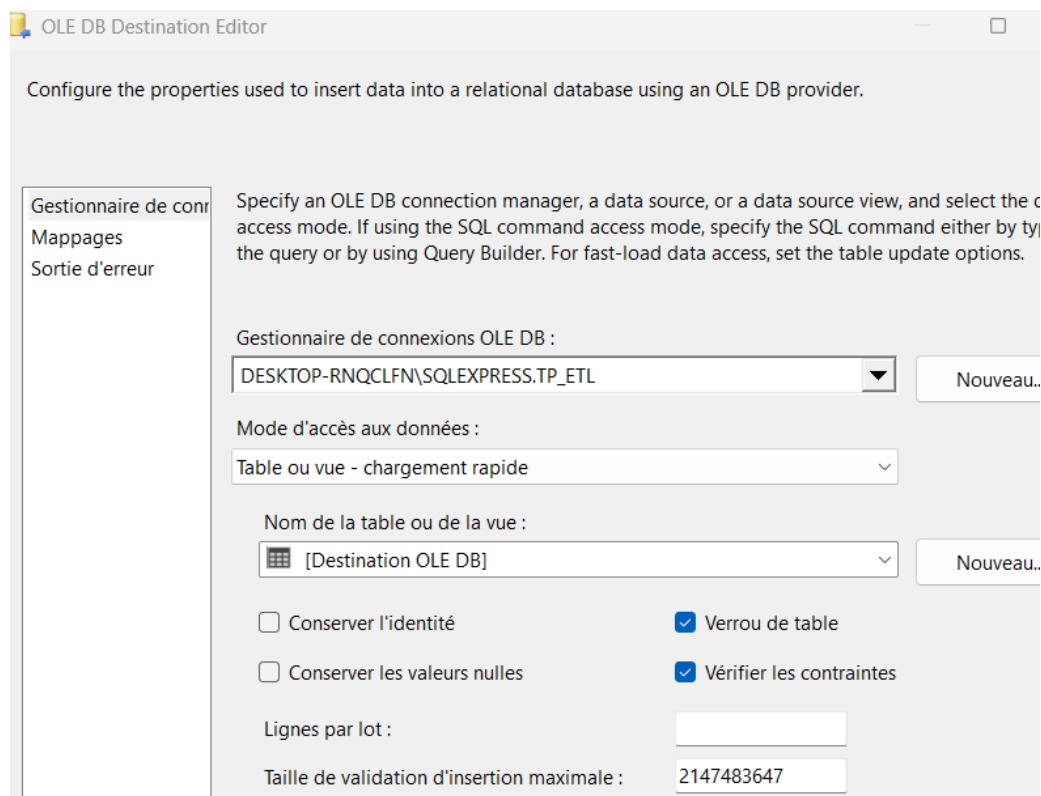


FIGURE 9.2 – OLE DB Destination Editor — configuration de la connexion et de la table cible

9.3 Mappages des Colonnes

L'onglet **Mappages** de l'éditeur de destination affiche la correspondance automatique entre les colonnes en entrée et les colonnes de destination. Les colonnes **Nom**, **DepartmentID**, **Name**, **GroupName** et **ModifiedDate** sont toutes mappées vers leurs équivalents dans la table cible.

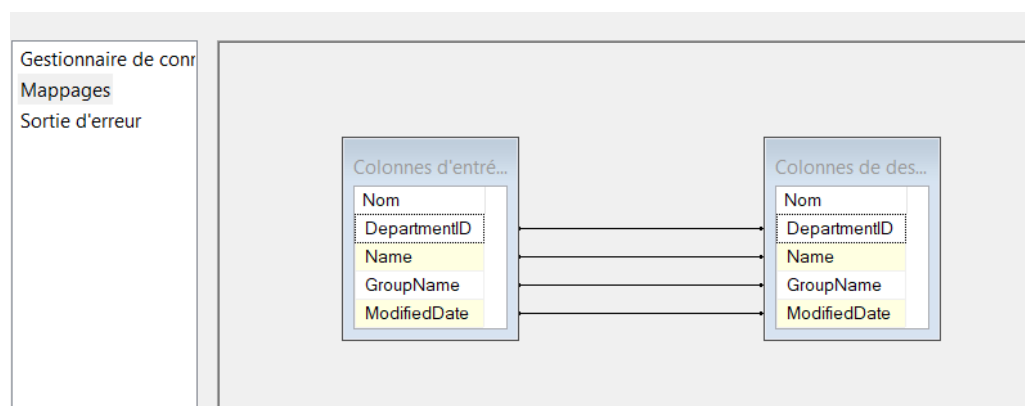


FIGURE 9.3 – Mappages des colonnes dans l'éditeur de la Destination OLE DB

9.4 Exécution du Flux de Données Simple

Après configuration, le flux de données est exécuté. Les deux composants **Source du fichier plat** et **Destination OLE DB** affichent des **icônes vertes**, confirmant le succès de la lecture et du chargement des données.

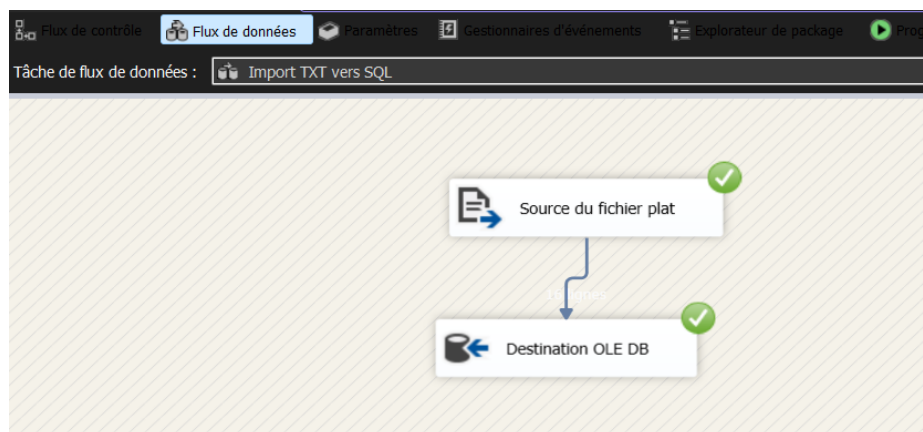


FIGURE 9.4 – Exécution réussie du flux de données — icônes vertes sur Source et Destination

Chapitre 10 TP 2 : Personnalisation —

Tâche SQL et Script Task

10.1 Ajout d'une Tâche d'Exécution SQL

Pour éviter les doublons à chaque exécution, une **Tâche d'exécution de requêtes SQL** est ajoutée dans le flux de contrôle, au-dessus de la tâche de flux de données. Une flèche verte de contrainte de précédence relie les deux tâches, garantissant que la suppression des données s'effectue avant l'importation.

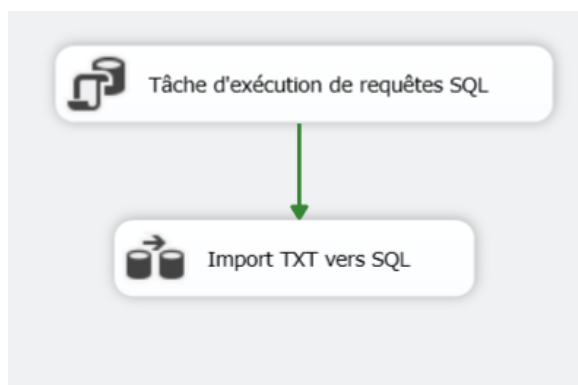


FIGURE 10.1 – Flux de contrôle : Tâche d'exécution SQL → Import TXT vers SQL

10.2 Ajout d'une Tâche de Script (Script Task)

Une **Tâche de script** est ajoutée en tête du flux de contrôle pour afficher un message de début d'exécution. Elle est liée à la tâche SQL, qui elle-même précède la tâche de flux de données. Le flux de contrôle complet est ainsi :

Tâche de script → Tâche d'exécution SQL → Import TXT vers SQL

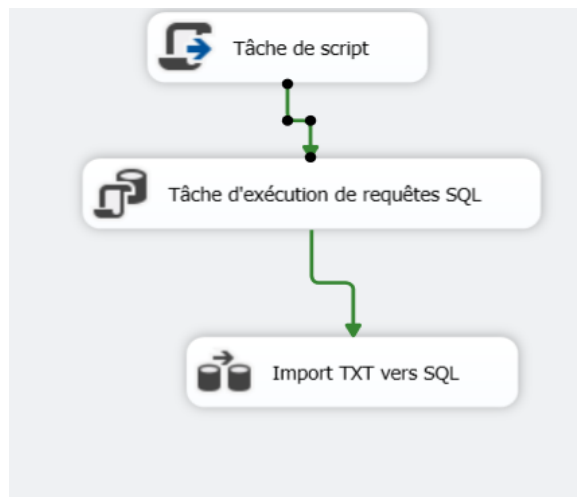


FIGURE 10.2 – Flux de contrôle complet : Script Task → Execute SQL Task → Data Flow Task

Le code C# inséré dans la tâche de script affiche une boîte de message :

```
MessageBox.Show("Mon premier prog ETL SSIS");
```


Chapitre 11 TP 2 : Colonne Dérivée dans le Flux de Données

11.1 Insertion d'un Composant Colonne Dérivée

Un composant **Colonne dérivée** est inséré entre la *Source du fichier plat* et la *Destination OLE DB*. Lors de l'exécution, les trois composants — Source, Colonne dérivée et Destination — affichent des **icônes vertes**, confirmant le bon fonctionnement de la transformation.

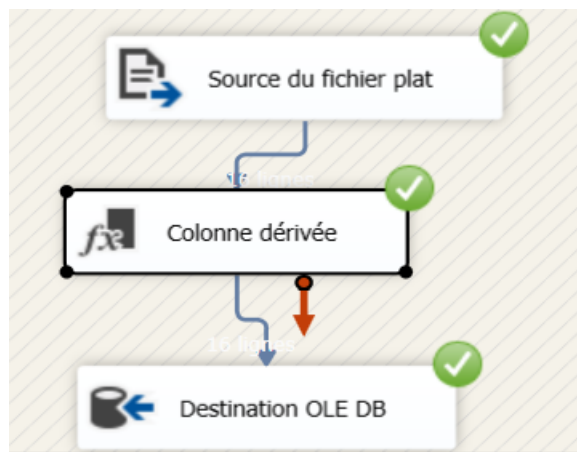


FIGURE 11.1 – Flux de données : Source → Colonne dérivée → Destination (exécution réussie)

La colonne dérivée permet d'appliquer des fonctions SQL sur les colonnes sources (par exemple, mettre le champ `Name` en majuscules via `UPPER(Name)`) avant de les charger dans la table de destination.

Chapitre 12 TP 2 : Chargement de Plusieurs Fichiers Plats — Conteneur Foreach

12.1 Objectif

Lorsque plusieurs fichiers texte doivent être importés dans une même table SQL Server, le **Conteneur de boucles Foreach** de SSIS permet d'itérer automatiquement sur chaque fichier d'un dossier et de le charger successivement.

12.2 Configuration du Conteneur Foreach

Un **Conteneur de boucles Foreach** est ajouté dans le flux de contrôle. À l'intérieur de ce conteneur, une **Tâche de flux de données** est placée. Une flèche verte de sortie indique que la boucle sera suivie d'une autre tâche.

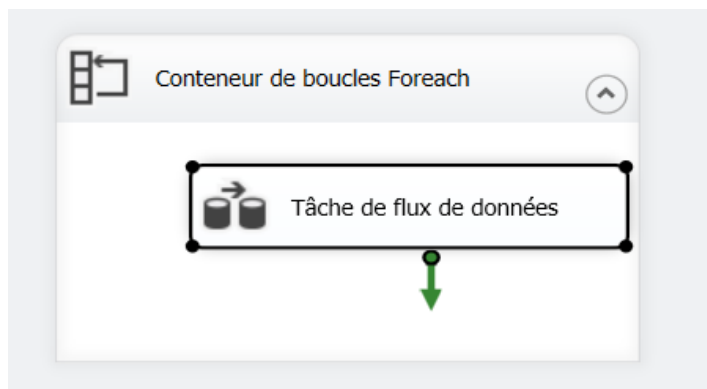


FIGURE 12.1 – Conteneur de boucles Foreach encapsulant la Tâche de flux de données

La configuration utilise l'énumérateur **Foreach File Enumerator**, pointant vers le dossier contenant les fichiers `*.txt`. Une variable de package `strFileName` est créée dans l'onglet *Variable Mappings* pour stocker dynamiquement le chemin de chaque fichier à chaque itération. La propriété `ConnectionString` du gestionnaire de connexion Fichier plat est ensuite liée à cette variable via l'éditeur d'expressions.

12.3 Flux de Données à l'Intérieur de la Boucle

Le flux de données interne comprend une **Source du fichier plat** reliée à une **Destination OLE DB**. Le chemin du fichier source est mis à jour dynamiquement à chaque itération grâce à la variable `strFileName`.

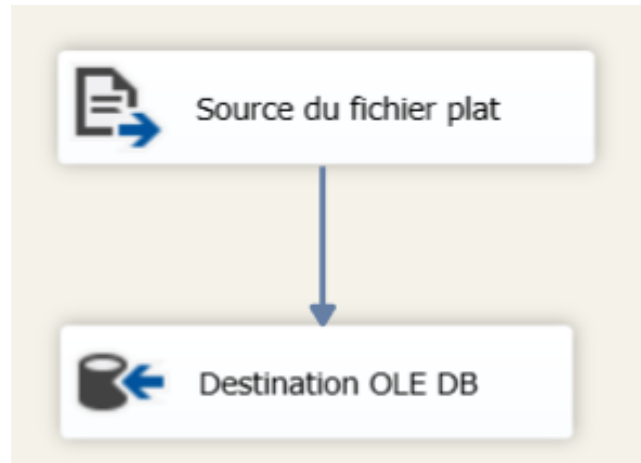


FIGURE 12.2 – Flux de données à l'intérieur du conteneur Foreach

12.4 Exécution Réussie de la Boucle

À l'exécution, les composants affichent des **icônes vertes** confirmant que tous les fichiers du dossier ont été chargés avec succès dans la table de destination.

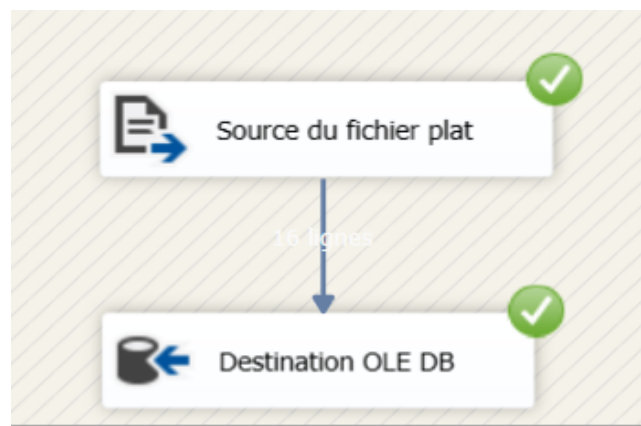


FIGURE 12.3 – Exécution réussie du flux dans la boucle Foreach

Chapitre 13 TP 2 : Extraction vers un Fichier Plat et Flux Multiples

13.1 Source OLE DB vers Destination Fichier Plat

En complément du flux principal, un flux inversé est créé : une **Source OLE DB** extrait les données de SQL Server et les charge dans une **Destination de fichier plat**. Ce flux illustre la capacité de SSIS à exporter des données vers des fichiers texte délimités.

Deux flux indépendants coexistent dans la même tâche de flux de données :

- Flux gauche : **Source Excel** → **Destination OLE DB** → **Destination OLE DB 1**
- Flux droit : **Source OLE DB** → **Destination de fichier plat**

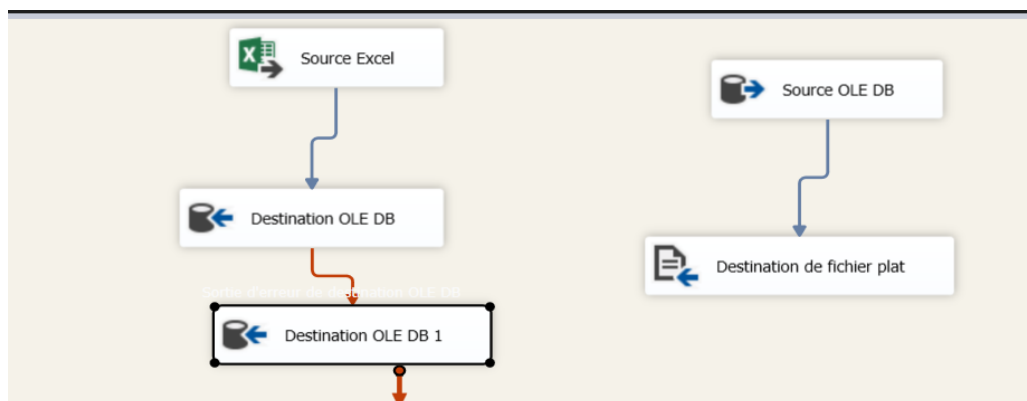


FIGURE 13.1 – Flux multiples : Source Excel → Destination OLE DB et Source OLE DB → Destination fichier plat

Chapitre 14 TP 2 : Gestion des Erreurs avec Event Handlers

14.1 Configuration du Gestionnaire d'Événements OnError

SSIS offre une gestion événementielle des erreurs via l'onglet **Gestionnaires d'événements**. L'événement **OnError** se déclenche automatiquement lors de toute erreur d'exécution dans le package. Une **Tâche de script** y est ajoutée pour intercepter et afficher le code et la description de l'erreur survenue.



FIGURE 14.1 – Gestionnaire d'événements OnError avec une Tâche de script

14.2 Configuration du Script de Gestion d'Erreur

Dans la tâche de script, les variables système `System::ErrorCode` et `System::ErrorDescription` sont déclarées en lecture seule. Le code C# inséré dans la fonction `Main` affiche le numéro et la description de l'erreur :

```
MessageBox.Show("Une erreur est survenue\nN: " +  
    Dts.Variables["ErrorCode"].Value.ToString() +  
    "\nDescription: " +  
    Dts.Variables["System::ErrorDescription"].Value.ToString());  
Dts.TaskResult = (int)ScriptResults.Success;
```

14.3 Types d'Erreurs Gérés

SSIS distingue trois catégories d'erreurs dans les flux de données :

- **Erreurs de conversion de données** : perte de chiffres significatifs ou troncation de chaînes lors des transformations.
- **Erreurs d'évaluation d'expressions** : opérations invalides ou valeurs manquantes dans les expressions SSIS.
- **Erreurs de recherche** : aucune correspondance retournée par une opération de recherche.

Pour chaque composant, le comportement en cas d'erreur peut être configuré selon trois modes : *Fail Component* (arrêt du package), *Ignore Failure* (l'erreur est ignorée) ou *Redirect Row* (la ligne erronée est redirigée vers une sortie d'erreur dédiée).

Chapitre 15 TP 3 : Introduction et Présentation du Projet BikeStores

15.1 Contexte et Objectifs

Ce troisième TP porte sur la **création et l'alimentation d'un entrepôt de données (Data Warehouse)** à partir d'une base de données relationnelle. La société fictive **BikeStores**, spécialisée dans la vente de vélos dans plusieurs villes, dispose d'un système opérationnel classique et souhaite mettre en place une solution décisionnelle pour améliorer la prise de décision.

Les objectifs de ce TP sont :

- Concevoir un Data Warehouse adapté aux besoins décisionnels de BikeStores.
- Mettre en œuvre le pipeline ETL d'alimentation du DW via SSIS.
- Respecter les contraintes d'intégrité référentielle lors du chargement des données.

15.2 Structure de la Base de Données Opérationnelle

La base de données **BikeStores** comprend deux schémas (**sales** et **production**) et neuf tables :

- **sales.stores** : informations sur les magasins
- **sales.staffs** : informations sur le personnel
- **sales.customers** : informations sur les clients
- **sales.orders** : en-têtes des commandes clients
- **sales.order_items** : lignes individuelles des commandes
- **production.categories** : catégories de vélos
- **production.brands** : marques de vélos
- **production.products** : informations sur les produits
- **production.stocks** : informations d'inventaire

15.3 Modélisation du Data Warehouse BikeStoresDW

Les dirigeants de BikeStores souhaitent analyser les ventes selon deux axes principaux :

- Quantité et montant des ventes en fonction du **store**, de la **ville** et de l'**état**.
- Quantité et montant des ventes en fonction de la **marque**, du **produit** et de la **catégorie**.

Le DW **BikeStoresDW** proposé est composé des tables suivantes :

Dimensions :

- **Dimension Store** avec la hiérarchie : Store → Ville → État
- **Dimension Produit** avec les hiérarchies : Produit → Marque et Produit → Catégorie

Table des faits :

- **Ventes** avec les mesures **Quantité** et **Montant**, reliée aux dimensions **Store** et **Produit**.

Chapitre 16 TP 3 : Nettoyage et Initialisation des Tables

16.1 Problématique de la Réexécution

Afin d'éviter la redondance des données lors de réexecutions multiples du package SSIS, il est nécessaire de vider les tables et de réinitialiser les clés primaires auto-incrémentées avant chaque chargement. L'ordre de suppression est crucial : il faut absolument commencer par la **table des faits** avant les tables de dimensions, pour respecter les contraintes d'intégrité référentielle.

16.2 Tâche d'Exécution SQL : Fact Vente

Une **Tâche d'exécution SQL** nommée *Fact Vente* est ajoutée dans le flux de contrôle. Elle exécute l'instruction suivante vers la base *BikeStoresDW* :

```
DELETE FROM [BikeStoresDW].[dbo].[ventes]
```

Deux tâches SQL supplémentaires sont ensuite ajoutées en parallèle, nommées **Dim Store** et **Dim Produit**, reliées en sortie de *Fact Vente*.

La tâche **Dim Store** exécute :

```
DELETE FROM [BikeStoresDW].[dbo].[Store]
DELETE FROM [BikeStoresDW].[dbo].[ville]
DELETE FROM [BikeStoresDW].[dbo].[etat]
DBCC CHECKIDENT (Store, RESEED, 0)
DBCC CHECKIDENT (ville, RESEED, 0)
DBCC CHECKIDENT (etat, RESEED, 0)
```

La tâche **Dim Produit** exécute :

```
DELETE FROM [BikeStoresDW].[dbo].[produit]
DELETE FROM [BikeStoresDW].[dbo].[categorie]
DELETE FROM [BikeStoresDW].[dbo].[marque]
DBCC CHECKIDENT (produit, RESEED, 0)
DBCC CHECKIDENT (categorie, RESEED, 0)
DBCC CHECKIDENT (marque, RESEED, 0)
```

Chapitre 17 TP 3 : Alimentation de la Dimension Store

17.1 Vue d'Ensemble du Flux de Contrôle Initial

Le flux de contrôle démarre avec les tâches de nettoyage (*Fact Vente*, *Dim Store*, *Dim Produit*), suivies des tâches de flux de données d'alimentation enchaînées séquentiellement. La tâche *Alimentation Etat* est la première tâche de flux de données reliée en sortie de *Dim Store*.

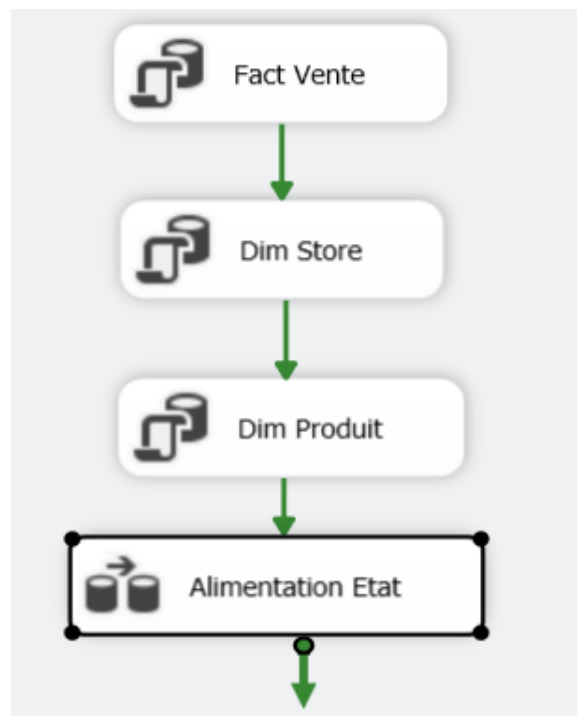


FIGURE 17.1 – Flux de contrôle : enchaînement des tâches d'alimentation de la Dimension Store

17.2 Alimentation de la Table État

La première tâche de flux de données alimente la table `etat` du DW. Elle comprend une **Source OLE DB** connectée à `BikeStores` et une **Destination OLE DB** pointant vers `BikeStoresDW`.

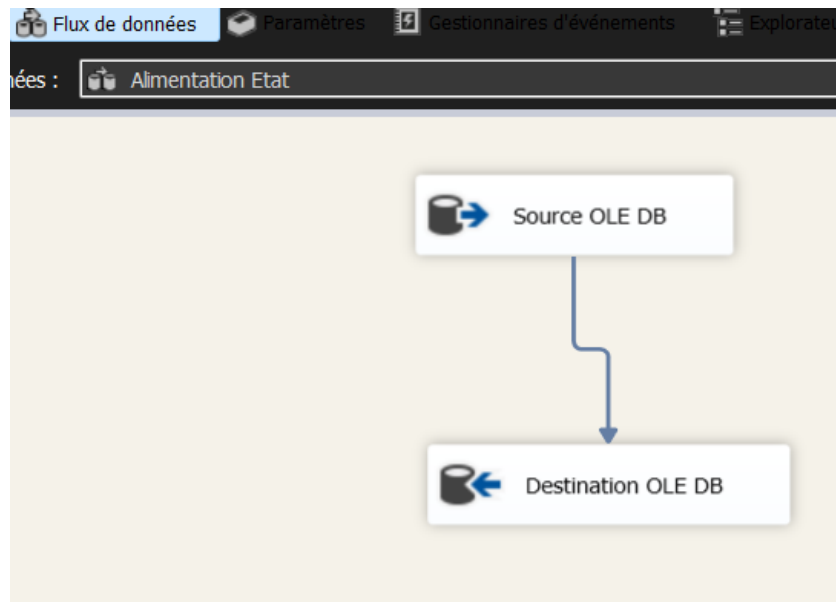


FIGURE 17.2 – Flux de données *Alimentation Etat* : Source OLE DB → Destination OLE DB

La requête SQL configurée dans la Source OLE DB extrait les états distincts depuis la table des magasins :

```
SELECT DISTINCT(state) FROM [BikeStores].[sales].[stores]
```

Le mapping configure `state` → `NomEtat`, tandis que `IDetat` est auto-incrémenté et ignoré en entrée.

17.3 Alimentation de la Table Ville

Une seconde tâche de flux de données alimente la table `ville`. Elle utilise une jointure entre la table `stores` de `BikeStores` et la table `etat` déjà remplie dans le DW :

```
SELECT s1.city, t2.IDetat
FROM [BikeStores].[sales].[stores] AS s1
CROSS JOIN [BikeStoresDW].[dbo].[etat] AS t2
WHERE s1.state = t2.NomEtat
```

Le mapping associe `city` → `NomVille` et `IDetat` → `IDetat`.

17.4 Alimentation de la Table Store

La troisième tâche alimente la table `Store` en joignant `BikeStores` et la table `ville` du DW :

```
SELECT s1.store_name, t2.IDville
FROM [BikeStores].[sales].[stores] AS s1
CROSS JOIN [BikeStoresDW].[dbo].[ville] AS t2
WHERE s1.city = t2.Nomville
```

Le mapping associe `store_name` $\rightarrow$ `Nomstore` et `IDville` $\rightarrow$ `IDVille`.

Chapitre 18 TP 3 : Alimentation de la Dimension Produit

18.1 Vue d'Ensemble du Flux de Contrôle Étendu

Après l'alimentation de la Dimension Store, le flux de contrôle s'étend pour intégrer l'alimentation en parallèle des tables **Marque** et **Catégorie**, suivie de l'alimentation de la table **Produit**.

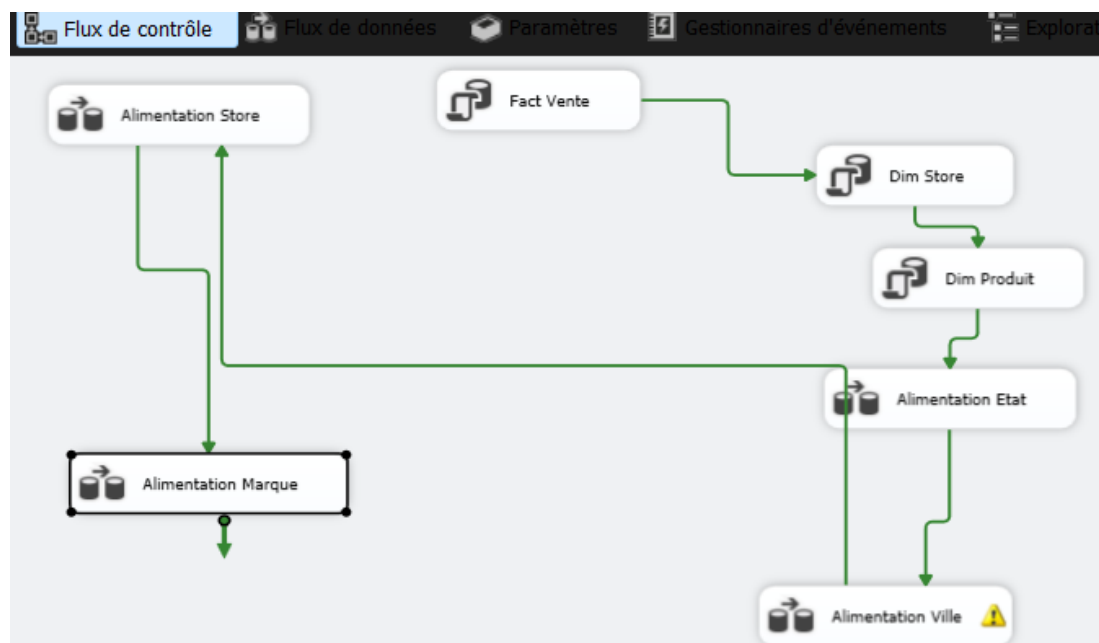


FIGURE 18.1 – Flux de contrôle global : alimentation des dimensions Store et Produit en parallèle

18.2 Alimentation des Tables Marque et Catégorie

Une tâche de flux de données contient deux flux parallèles indépendants : l'un pour la table **Marque** et l'autre pour la table **Catégorie**.

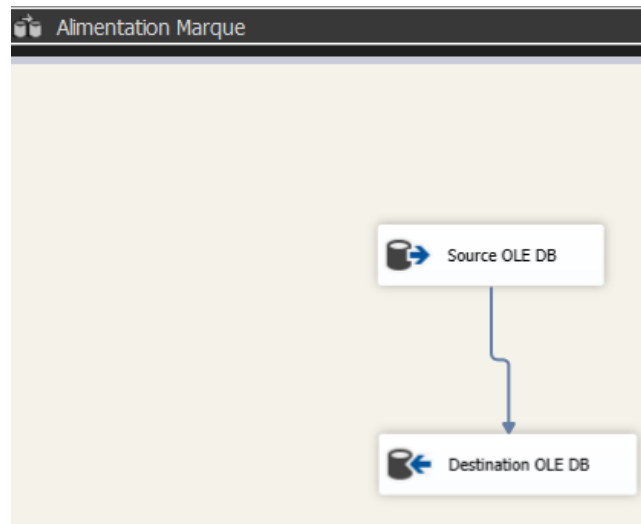


FIGURE 18.2 – Flux *Alimentation Marque* : Source OLE DB → Destination OLE DB

Pour la table **Marque**, la requête source est :

```
SELECT [brand_name]
FROM [BikeStores].[production].[brands]
```

Le mapping associe `brand_name` → `marque`.

Pour la table **Catégorie**, la requête source est :

```
SELECT [category_name]
FROM [BikeStores].[production].[categories]
```

Le mapping associe `category_name` → `caategorie`.

18.3 Alimentation de la Table Produit

Une tâche de flux de données dédiée alimente la table **Produit** en effectuant des jointures entre la table `products` de `BikeStores` et les tables **Catégorie** et **marque** du DW :

```
SELECT p.product_name, c.IDcategorie, m.IDMarque
FROM [BikeStores].[production].[products] AS p
CROSS JOIN [BikeStoresDW].[dbo].[Categorie] AS c
CROSS JOIN [BikeStoresDW].[dbo].[marque] AS m
WHERE p.brand_id = m.IDMarque
      AND p.category_id = c.IDcategorie
```

Le mapping associe : `product_name` → `nomProduit`, `IDMarque` → `IDmarque`, `IDcategorie` → `IDcategorie`.

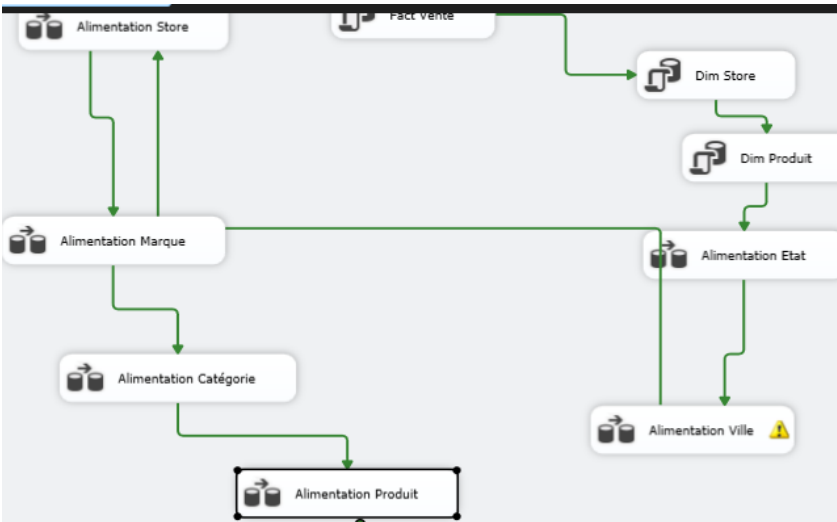


FIGURE 18.3 – Flux *Alimentation Catégorie* : Source OLE DB → Destination OLE DB

Chapitre 19 TP 3 : Alimentation de la Table des Faits Ventes

19.1 Flux de Contrôle Final

Une fois toutes les dimensions alimentées, la tâche de flux de données d'alimentation de la **table des faits Ventes** est ajoutée. Elle reçoit les contraintes de précedence depuis les deux branches (Dimension Store et Dimension Produit) pour garantir que toutes les clés étrangères existent avant l'insertion.

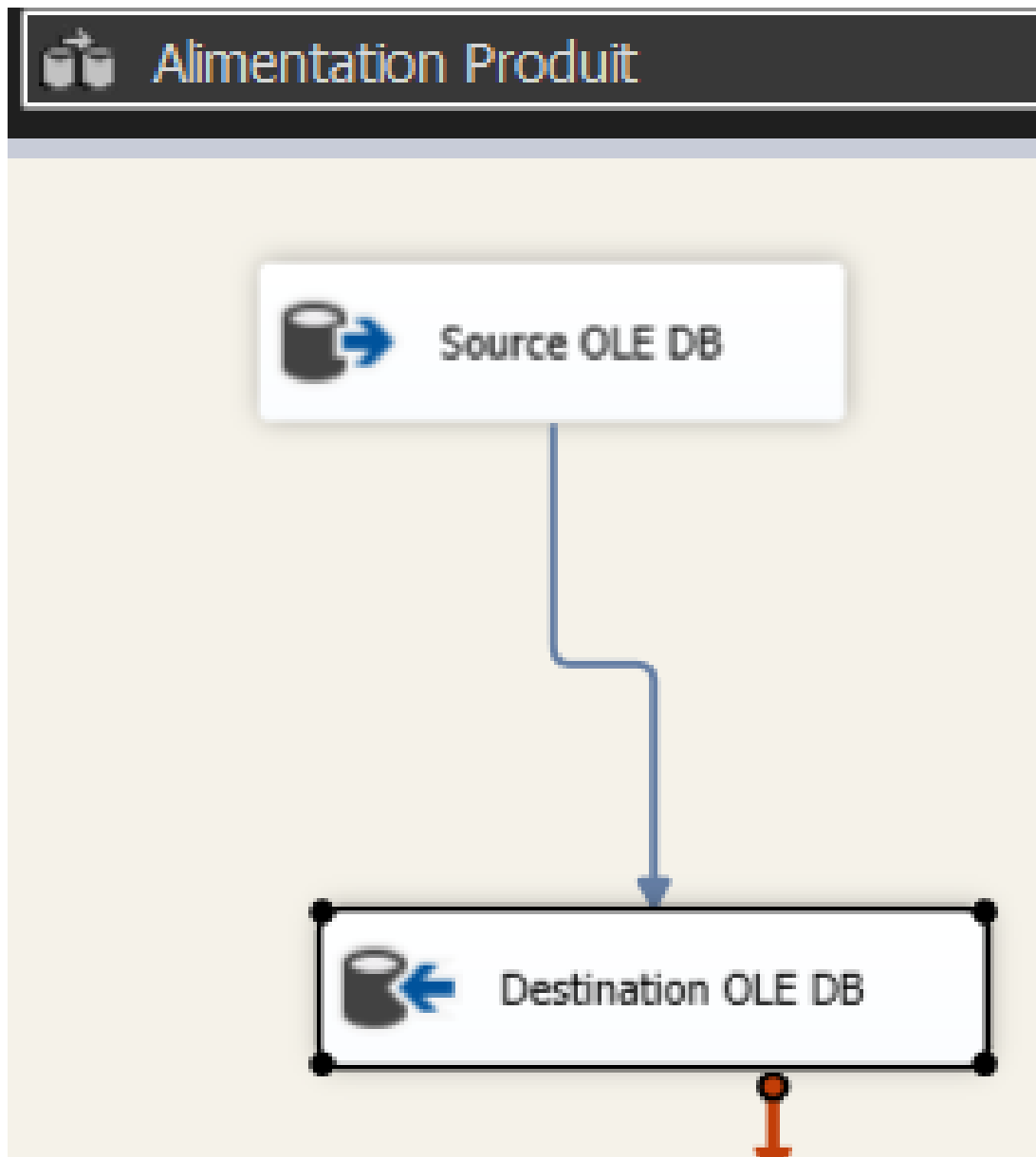


FIGURE 19.1 – Flux de contrôle complet incluant l’alimentation de la table des faits Ventes

19.2 Configuration du Flux de Données Ventes

Le flux de données contient une **Source OLE DB** connectée à **BikeStoresDW** et à **BikeStores**, et une **Destination OLE DB** vers la table **Ventes** du DW.

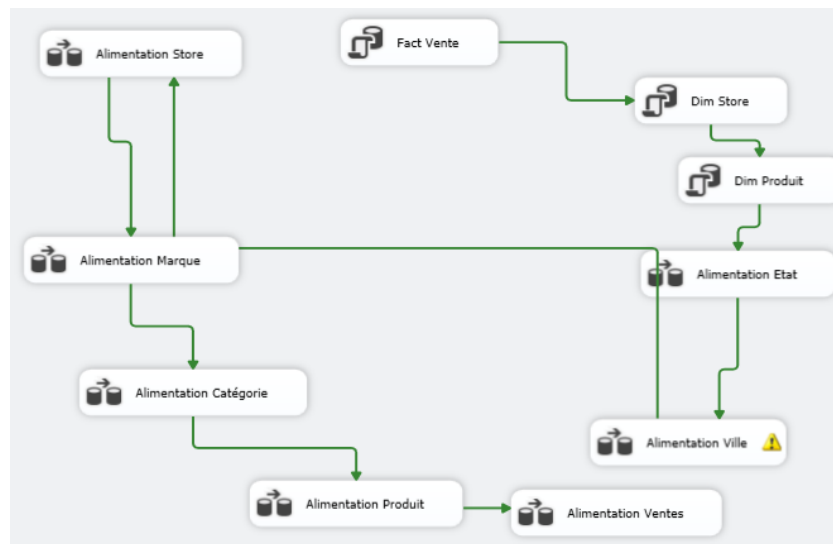


FIGURE 19.2 – Flux *Alimentation Ventes* : Source OLE DB → Destination OLE DB

La requête SQL d'alimentation de la table des faits est :

```

SELECT DISTINCT oi.product_id
    ,oi.[quantity]
    ,s.IDstore
    ,oi.[list_price]*oi.[quantity] AS montant
FROM [BikeStores].[sales].[order_items] AS oi
CROSS JOIN [BikeStores].[sales].[orders] AS o
CROSS JOIN [BikeStoresDW].[dbo].[store] AS s
WHERE oi.order_id = o.order_id
    AND o.store_id = s.IDstore
ORDER BY oi.product_id

```

Le mapping final associe : quantity → Quantite, montant → Montant, IDstore → IDStore, product_id → IDProduit.

Chapitre 20 TP 3 : Exécution Finale du Package BikeStoresDW

20.1 Résultat de l'Exécution Complète

Après la configuration de l'ensemble des tâches, le package est exécuté. Toutes les tâches affichent des **icônes vertes**, confirmant le succès de chaque étape du pipeline ETL.

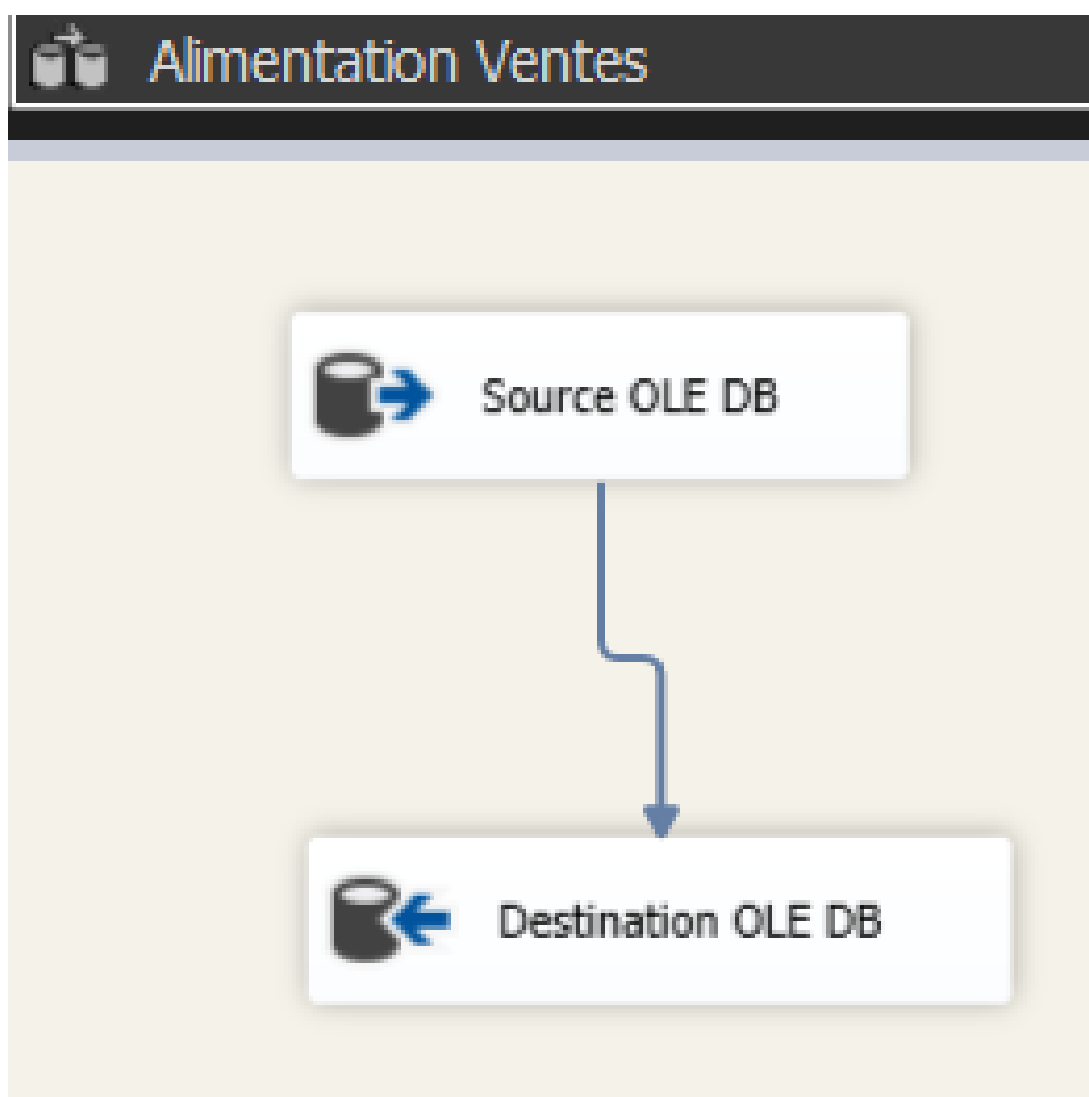


FIGURE 20.1 – Exécution réussie du package — toutes les tâches affichent une icône verte

L'ordre d'exécution respecté est le suivant :

1. **Fact Vente** : suppression de la table des faits.
2. **Dim Store** et **Dim Produit** : suppression et réinitialisation des dimensions (en parallèle).
3. **Alimentation Etat** → **Alimentation Ville** → **Alimentation Store** : chargement séquentiel de la Dimension Store.
4. **Alimentation Marque** + **Alimentation Catégorie** (en parallèle) → **Alimentation Produit** : chargement de la Dimension Produit.
5. **Alimentation Ventes** : chargement final de la table des faits.

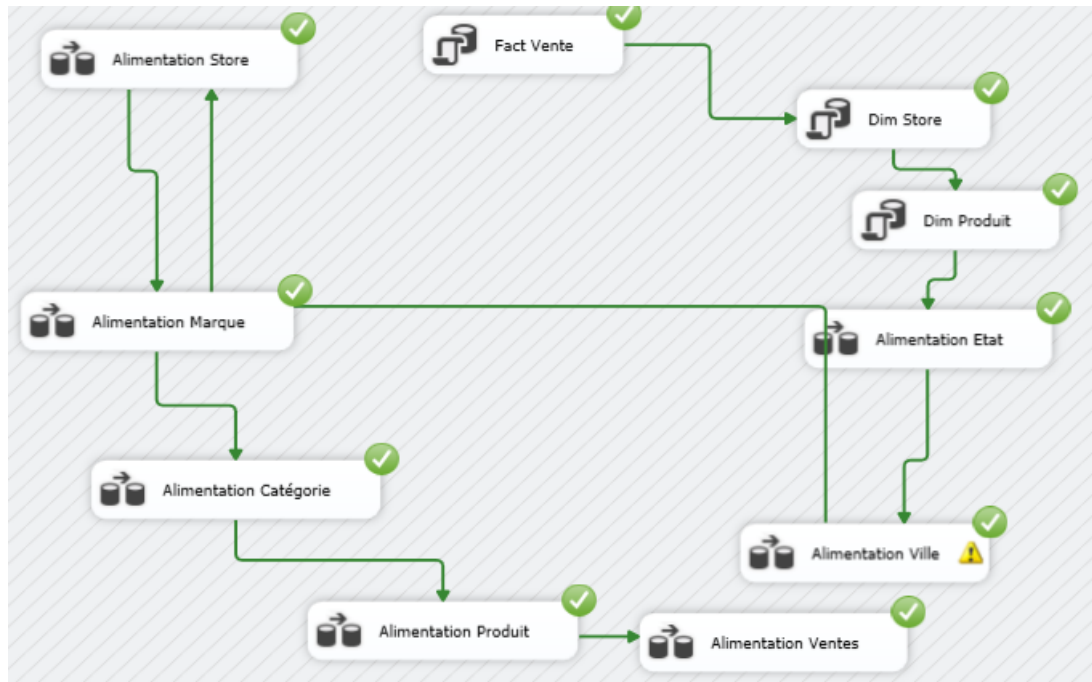


FIGURE 20.2 – Vue globale du flux de contrôle complet après exécution réussie

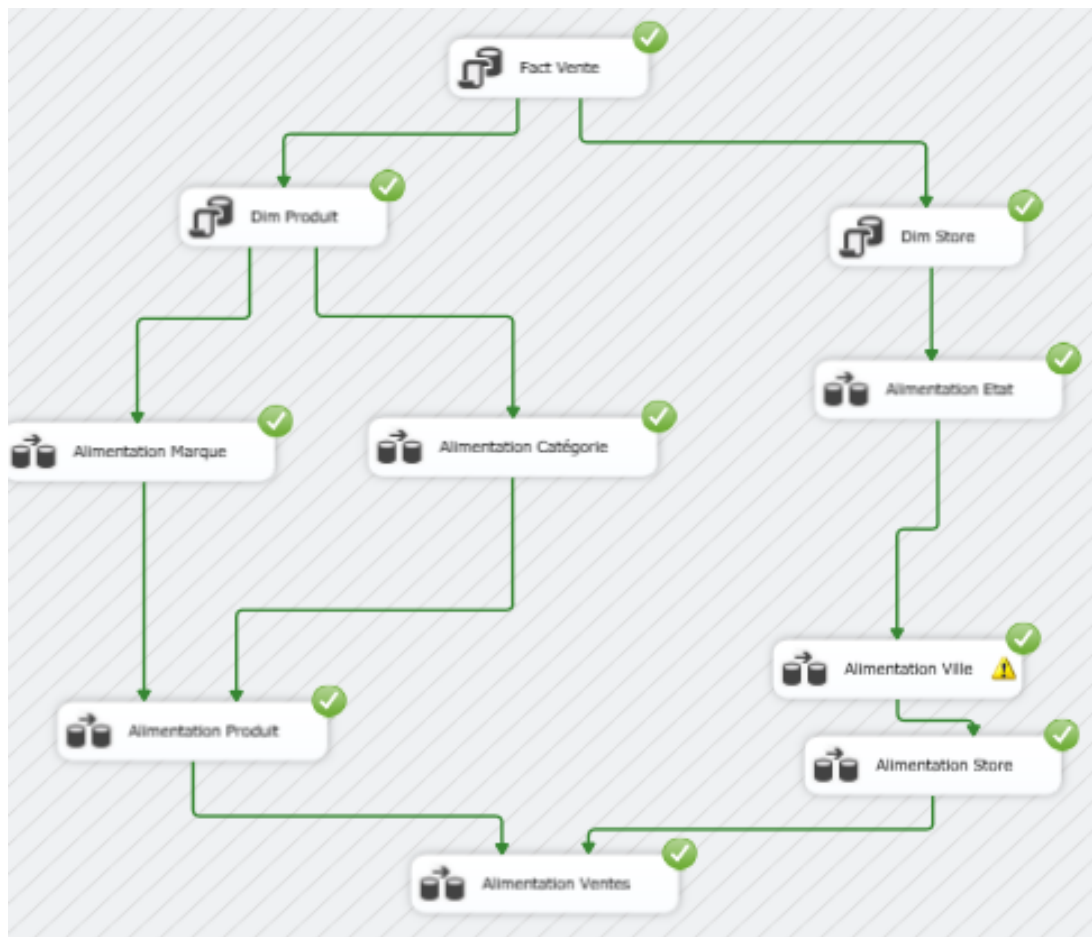


FIGURE 20.3 – Vue globale 2 du flux de contrôle complet après exécution réussie

Partie II : Visualisation avec Microsoft Power BI

Les travaux pratiques suivants portent sur **Microsoft Power BI**, outil de visualisation et d'analyse décisionnelle de la suite Microsoft BI. Power BI permet de connecter des sources de données variées, de modéliser les données et de créer des tableaux de bord interactifs riches pour l'aide à la décision.

Chapitre 21 TP Power BI 1 : Tableau de Bord Superstore

21.1 Présentation du Projet

Ce premier TP Power BI porte sur l'analyse des données de ventes d'une entreprise de distribution, à partir d'un jeu de données de type *Superstore*. L'objectif est de créer un tableau de bord interactif multi-pages permettant d'analyser les ventes, les bénéfices et les quantités vendues selon plusieurs dimensions : temporelle, géographique, par segment client et par catégorie de produits.

21.2 Page 1 — Vue Globale des Ventes

La première page du rapport présente une vue synthétique des indicateurs clés de performance (KPI) et plusieurs visualisations croisées.

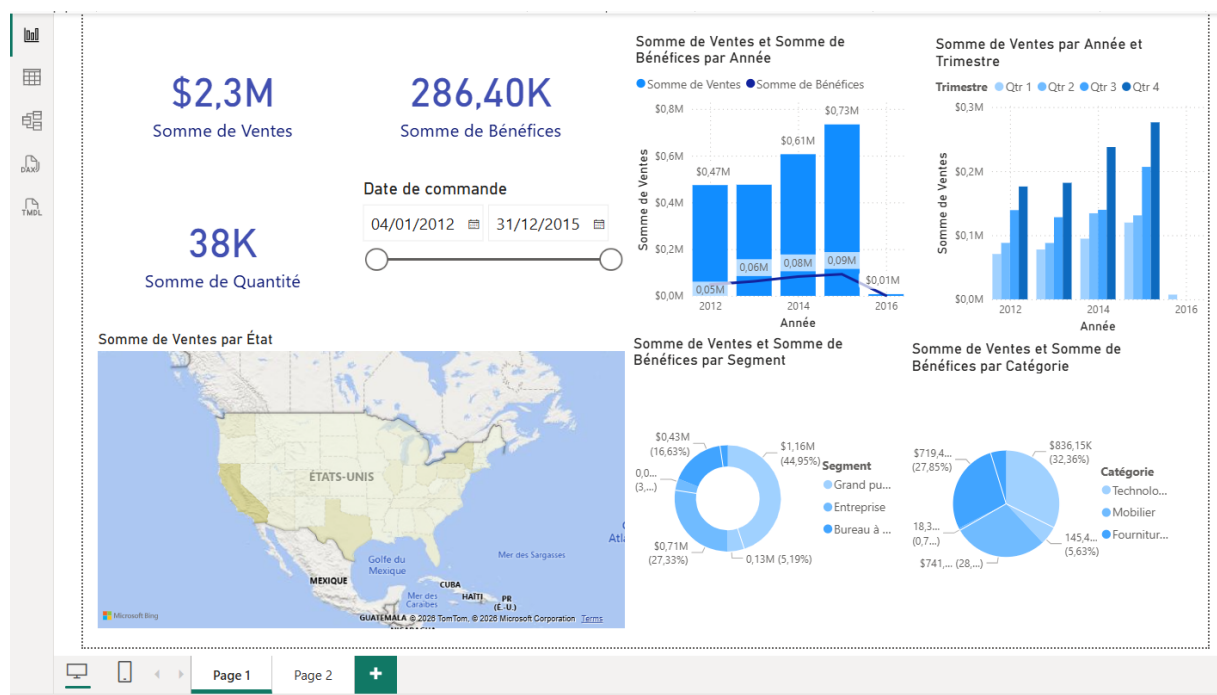


FIGURE 21.1 – Page 1 du tableau de bord Superstore — Vue globale des ventes et bénéfices

Les éléments présents sur cette page sont :

- **Cartes KPI :** Somme de Ventes (\$2,3M), Somme de Bénéfices (286,40K) et Somme de Quantité (38K), affichées en haut à gauche pour une lecture immédiate.

- **Segment temporel interactif** : un curseur de dates permet de filtrer la période d'analyse (ici du 04/01/2012 au 31/12/2015).
- **Graphique en barres groupées** : Somme de Ventes et Somme de Bénéfices par Année, permettant de visualiser l'évolution temporelle et comparer les deux mesures.
- **Graphique en barres empilées par trimestre** : Somme de Ventes par Année et Trimestre, avec légende colorée par trimestre (Qtr 1 à Qtr 4).
- **Carte géographique** : Somme de Ventes par État, affichant la répartition géographique des ventes sur la carte des États-Unis via Microsoft Bing Maps.
- **Graphiques en anneau** : Somme de Ventes et Bénéfices par Segment client (Grand public, Entreprise, Bureau à domicile) et par Catégorie de produits (Technologie, Mobilier, Fournitures).

21.3 Page 2 — Analyse Détaillée par Région et Catégorie

La deuxième page offre une analyse plus granulaire avec des filtres et des visualisations croisées par région, état, ville et catégorie de produits.

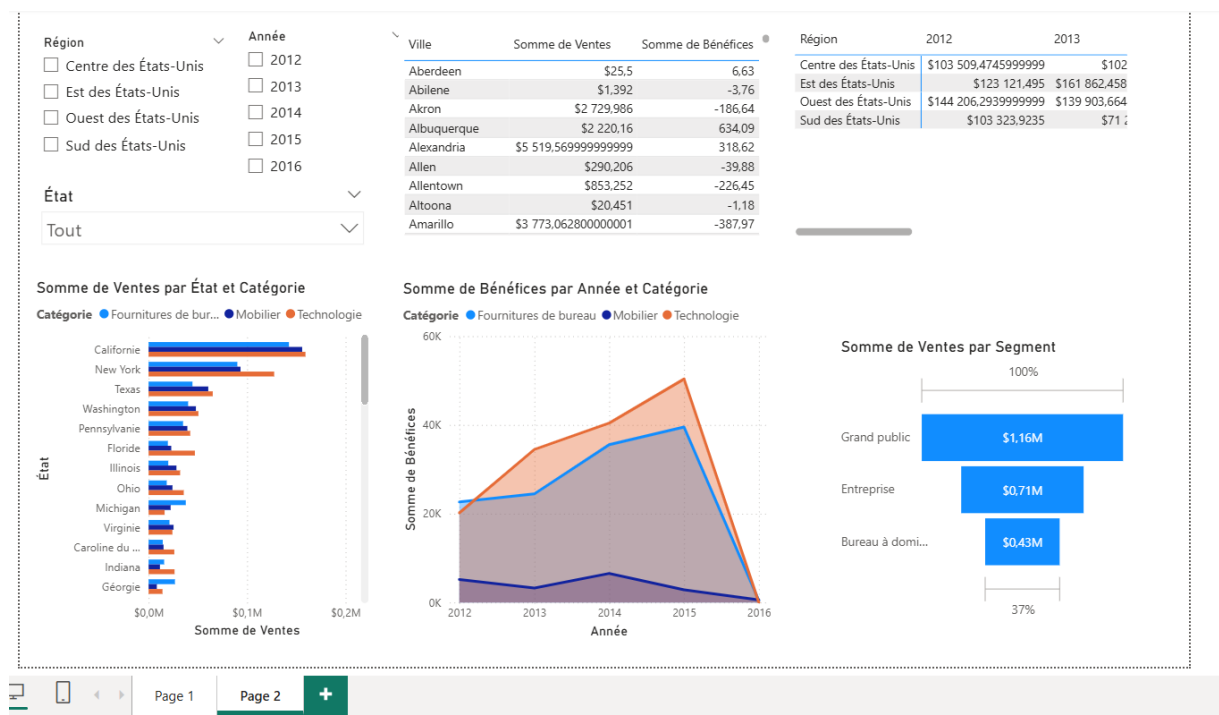


FIGURE 21.2 – Page 2 du tableau de bord Superstore — Analyse détaillée par région et catégorie

Les éléments présents sur cette page sont :

- **Segments filtrés** : filtres interactifs par Région (Centre, Est, Ouest, Sud des États-Unis) et par Année (2012 à 2016), ainsi qu'un filtre déroulant par État.
- **Tableau de données** : liste des villes avec leur Somme de Ventes et Somme de Bénéfices correspondantes.
- **Tableau croisé Région/Année** : matrice affichant les ventes par région et par année pour une comparaison temporelle et géographique.
- **Graphique en barres horizontal** : Somme de Ventes par État et Catégorie, permettant d'identifier les États les plus performants (Californie, New York, Texas en tête).
- **Graphique en aires** : Somme de Bénéfices par Année et Catégorie (Fournitures de bureau, Mobilier, Technologie), illustrant les tendances de rentabilité par catégorie.

- **Graphique en barres horizontales** : Somme de Ventes par Segment (Grand public 1,16M\$, Entreprise 0,71M\$, Bureau à domicile 0,43M\$).

Chapitre 22 TP Power BI 2 : Tableau de Bord import-export

22.1 Présentation du Projet

Ce deuxième TP Power BI porte sur l'analyse des données d'une base de données d'import-export de produits alimentaires. Le tableau de bord créé permet d'analyser le chiffre d'affaires, le nombre de produits et de clients, ainsi que la répartition des ventes par catégorie, par pays et par produit.

22.2 Tableau de Bord import-export

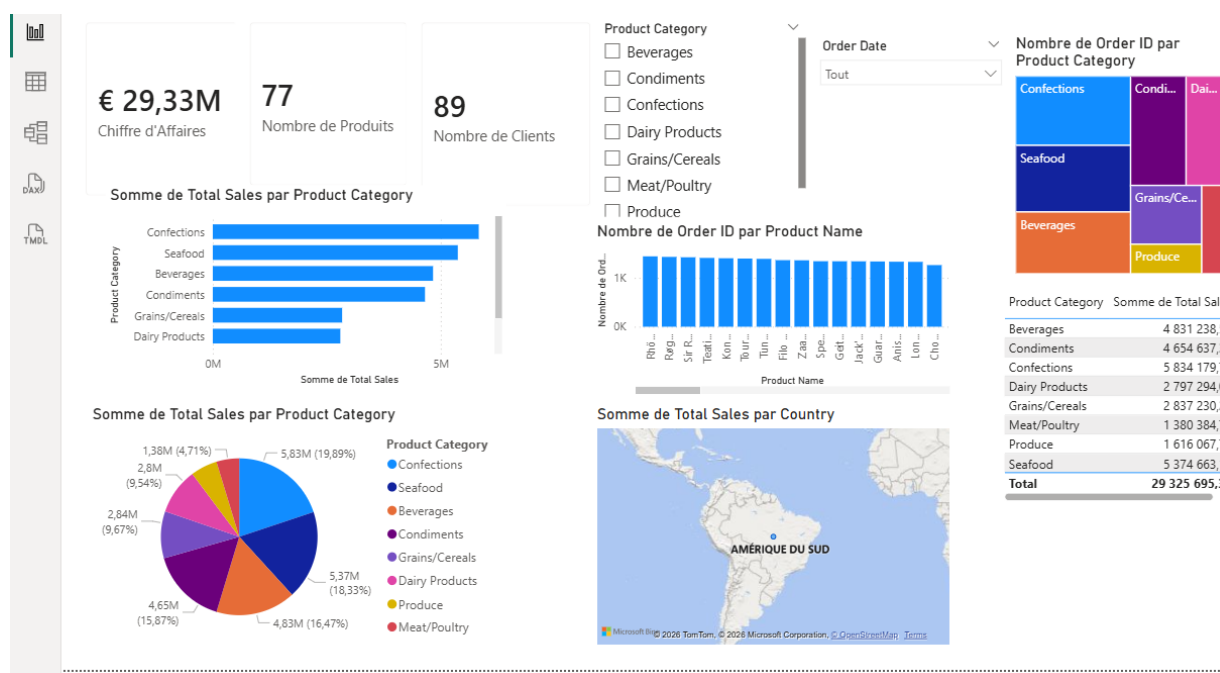


FIGURE 22.1 – Tableau de bord Power BI — Analyse des ventes Northwind

Les éléments présents dans ce tableau de bord sont :

- **Cartes KPI** : Chiffre d'Affaires total (29,33M€), Nombre de Produits (77) et Nombre de Clients (89).
- **Filtres interactifs** : segments par *Product Category* (Beverages, Condiments, Confections, Dairy Products, Grains/Cereals, Meat/Poultry, Produce) et par *Order Date*.
- **Graphique en barres horizontal** : Somme de Total Sales par Product Category, permettant d'identifier les catégories les plus vendues (Confections et Seafood en tête).

- **Graphique en barres groupées** : Nombre de Order ID par Product Name, affichant le volume de commandes pour chaque produit.
- **Graphique en secteurs (camembert)** : répartition de la Somme de Total Sales par Product Category avec les pourcentages correspondants.
- **Treemap** : Nombre de Order ID par Product Category, offrant une vue hiérarchique de la contribution de chaque catégorie au volume de commandes.
- **Tableau récapitulatif** : liste des catégories avec leur Somme de Total Sales respective, avec le total global de 29 325 695,3€.
- **Carte géographique** : Somme de Total Sales par Country, affichant la distribution géographique des ventes dans le monde.

Chapitre 23 TP Power BI 3 : Tableau de Bord Stocks Restauration

23.1 Présentation du Projet

Ce troisième TP Power BI porte sur l'analyse des stocks d'une entreprise de restauration. Le tableau de bord permet de suivre les quantités en stock par famille de produits (viande, boissons, etc.), d'analyser les tendances mensuelles et d'identifier les produits les plus représentés dans le stock.

23.2 Tableau de Bord Stocks

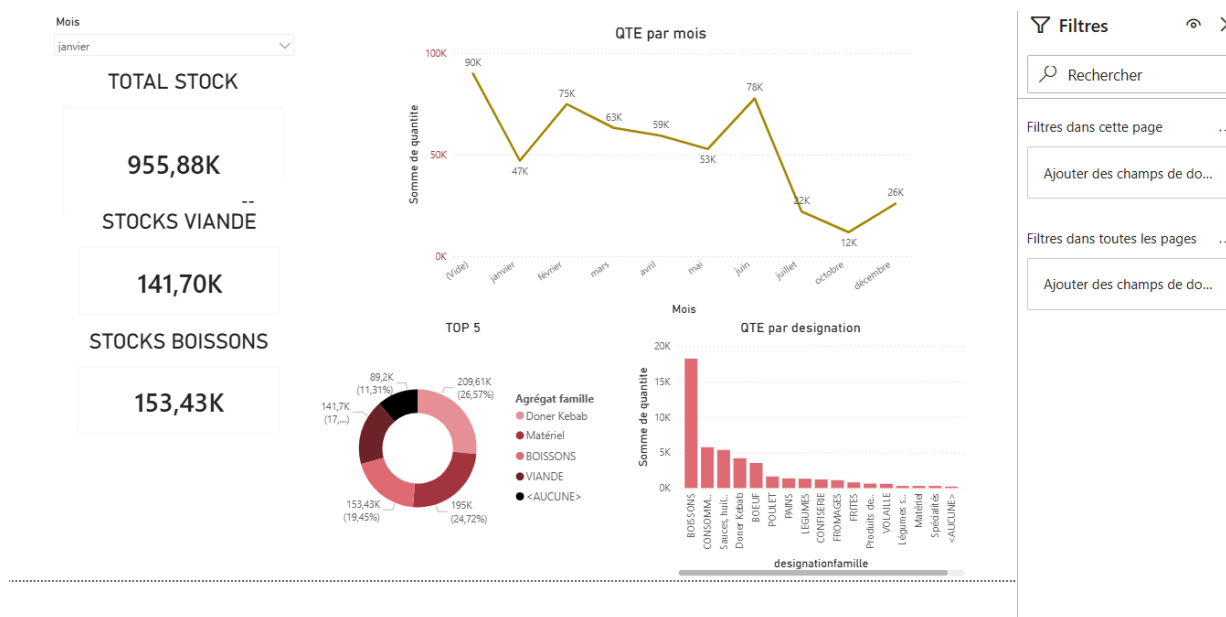


FIGURE 23.1 – Tableau de bord Power BI — Analyse des stocks de restauration

Les éléments présents dans ce tableau de bord sont :

- **Filtre par Mois** : un menu déroulant (ici sélectionné sur *janvier*) permet de filtrer dynamiquement toutes les visualisations par mois.
- **Cartes KPI** :
 - **TOTAL STOCK** : 955,88K unités au total.
 - **STOCKS VIANDE** : 141,70K unités.
 - **STOCKS BOISSONS** : 153,43K unités.

- **Graphique en courbes** : QTE par mois, affichant l'évolution des quantités en stock tout au long de l'année (de janvier à décembre), avec un pic en janvier (90K) et un creux en novembre (12K).
- **Graphique en anneau (TOP 5)** : répartition des stocks par famille agrégée (Doner Kebab 26,57%, BOISSONS 19,45%, Matériel 24,72%, VIANDE 17%, autres), permettant d'identifier les familles dominantes.
- **Graphique en barres vertical** : QTE par designation, présentant le détail des quantités par désignation de famille de produits (BOISSONS, CONSOMM., Sauces/huil., ROEUF, Doner Kebab, POULET, etc.).
- **Panneau Filtres** : panneau latéral droit affichant les filtres appliqués dans la page et dans toutes les pages du rapport.

Chapitre 24 TP Power BI 4 : Tableau de Bord Retail Australie

24.1 Présentation du Projet

Ce quatrième et dernier TP Power BI porte sur l'analyse des ventes au détail d'une chaîne de distribution australienne. Le projet met en œuvre un modèle de données en étoile (*star schema*) dans Power BI avec plusieurs tables de dimensions et une table de faits, permettant des analyses croisées avancées par chaîne, état, catégorie, manager et trimestre fiscal.

24.2 Modèle de Données en Étoile

Avant de créer les visualisations, un modèle de données relationnel est défini dans Power BI. Il comprend une table de faits centrale reliée à plusieurs dimensions :

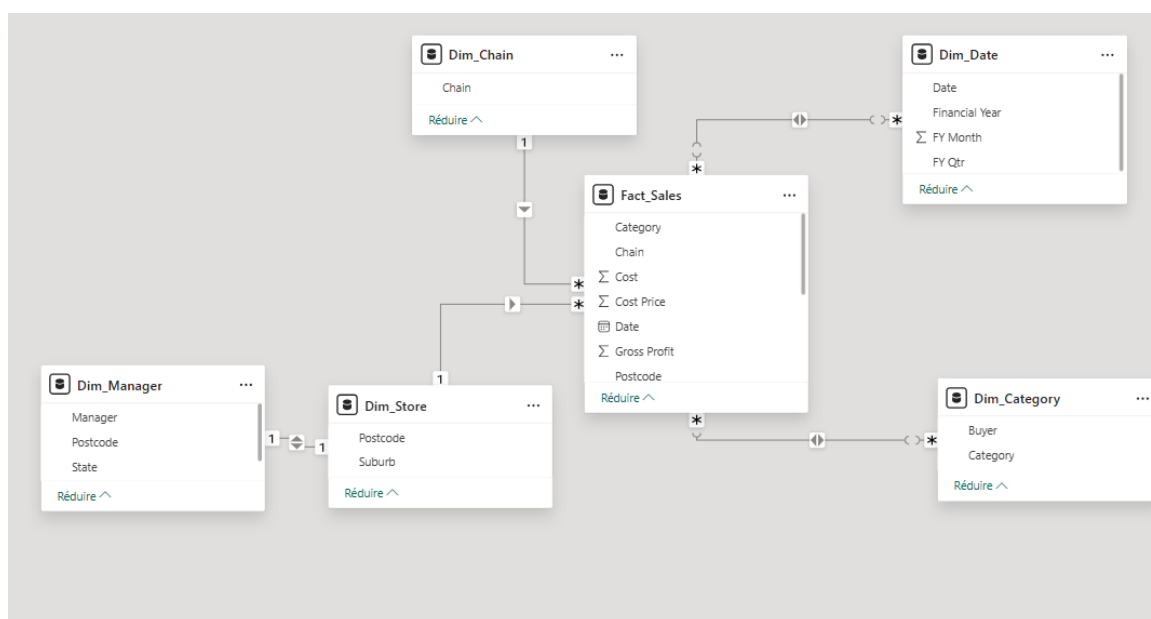


FIGURE 24.1 – Modèle de données en étoile dans Power BI — Fact_Sales et ses dimensions

Le modèle est composé des tables suivantes :

- **Fact_Sales** (table centrale) : contient les mesures Category, Chain, Cost, Cost Price, Date, Gross Profit, Postcode, ainsi que les clés étrangères vers les dimensions.
- **Dim_Date** : dimension temporelle avec Date, Financial Year, FY Month, FY Qtr.
- **Dim_Chain** : dimension chaîne de magasins (Chain).
- **Dim_Store** : dimension magasin avec Postcode et Suburb.

- **Dim_Manager** : dimension manager avec Manager, Postcode et State.
- **Dim_Category** : dimension catégorie avec Buyer et Category.

Les relations sont de type un-à-plusieurs (1 vers *) entre chaque dimension et la table de faits, conformément au schéma en étoile.

24.3 Page 1 — Vue Globale des Ventes par Chaîne et État

La première page du rapport présente une analyse croisée des ventes par chaîne de magasins (*Bellings* et *Ready Wear*), par état australien et par catégorie de produits.



FIGURE 24.2 – Page 1 du tableau de bord Retail Australia — Ventes par chaîne, état et catégorie

Les éléments présents sur cette page sont :

- **Filtre par State** : segment permettant de filtrer par état australien (ACT, NSW, NT, QLD, SA, TAS, VIC, WA).
- **Graphique en courbes** : Sales par Date, affichant l'évolution temporelle des ventes de 2016 à 2017.
- **Graphique en barres groupées** : Sales par State et Chain, comparant les ventes des deux chaînes (Bellings en rouge, Ready Wear en jaune) pour chaque état — NSW et QLD dominant.
- **Graphique en secteurs** : Sales par Chain, montrant que Ready Wear représente 70,9% des ventes (43,14M€) contre 29,1% pour Bellings (17,7M€).
- **Graphique en barres horizontales** : Sales par Category et Chain, permettant de comparer la performance des deux chaînes pour chaque catégorie (Mens, Shoes, Juniors, Home, Kids, Womens, Accessories, Intimate, Groceries, Hosiery).
- **Graphique combiné barres/courbe** : Sales et Gross Profit par FY Qtr, avec les barres grises pour les ventes et la courbe noire pour le Gross Profit par trimestre fiscal.

- **Graphique à bulles animé** : Sales, GP% et Somme de Gross Profit par Category et FY Qtr, permettant une analyse dynamique de la rentabilité par catégorie et trimestre.
- **Carte géographique** : Sales par State, affichant la répartition géographique des ventes en Australie.

24.4 Page 2 — Analyse par Manager et Suburb

La deuxième page approfondit l'analyse en ajoutant les dimensions Manager et Suburb (banlieue/quartier), permettant une analyse de la performance commerciale au niveau des responsables et de la localisation des magasins.

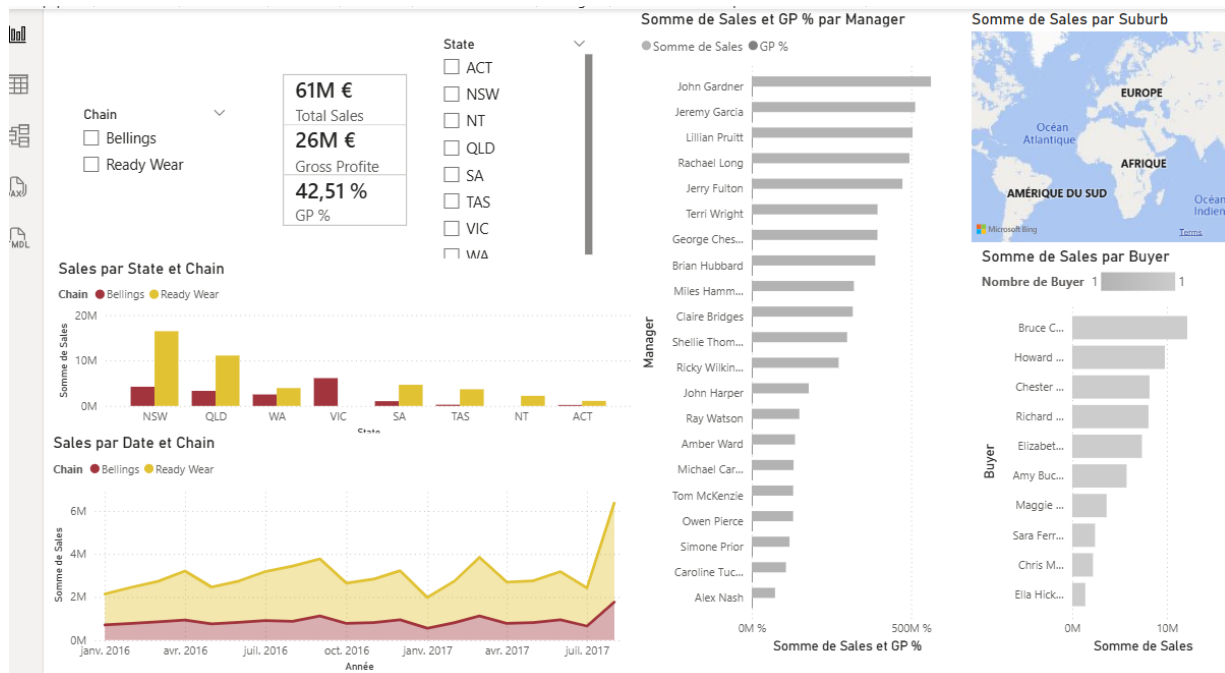


FIGURE 24.3 – Page 2 du tableau de bord Retail Australia — Analyse par manager, buyer et suburb

Les éléments présents sur cette page sont :

- **Filtres interactifs** : segments par Chain (Bellings, Ready Wear) et par State (ACT, NSW, NT, QLD, SA, TAS, VIC, WA).
- **Cartes KPI** : Total Sales (61M€), Gross Profit (26M€) et GP% (42,51%).
- **Graphique en barres groupées** : Sales par State et Chain, offrant la même vue comparative que la page 1 mais dans le contexte des filtres de cette page.
- **Graphique en aires** : Sales par Date et Chain, affichant l'évolution temporelle des ventes des deux chaînes de janvier 2016 à juillet 2017.
- **Graphique en barres horizontal** : Somme de Sales et GP% par Manager, classant les managers par performance commerciale totale (John Gardner en tête).
- **Carte géographique** : Somme de Sales par Suburb, localisant les magasins sur la carte mondiale.
- **Graphique en barres horizontal avec curseur** : Somme de Sales par Buyer, filtrable par nombre de buyers, permettant d'analyser la performance des acheteurs (Bruce C., Howard, Chester, Richard, Elizabeth, Amy, Maggie, Sara, Chris, Ella en tête).

Conclusion

L'ensemble de ces travaux pratiques a permis de couvrir le cycle complet d'un projet de **Business Intelligence**, depuis l'intégration des données jusqu'à leur visualisation et exploitation décisionnelle. Étape par étape, nous avons :

- Créé et configuré des gestionnaires de connexions OLE DB vers des bases hétérogènes (`gestion_livres`, `TP_ETL`, `BikeStores`, `BikeStoresDW`), ainsi que des connexions vers des fichiers plats et Excel.
- Construit des flux de données SSIS avec des composants Source OLE DB, Colonne dérivée et Destination OLE DB, enrichis de colonnes calculées via des expressions conditionnelles.
- Intégré des tâches d'exécution SQL pour vider et réinitialiser les tables avant chaque chargement, évitant ainsi les doublons et les conflits de clés primaires.
- Automatisé le chargement de plusieurs fichiers texte via le **Conteneur de boucles Foreach** et géré proactivement les erreurs via l'**Event Handler OnError**.
- Conçu et alimenté un **Data Warehouse en étoile** (`BikeStoresDW`) à partir d'une base de données relationnelle, en respectant strictement l'ordre d'alimentation : table des faits → dimensions → table des faits.
- Exploité **Microsoft Power BI** pour créer des tableaux de bord interactifs et multi-pages à partir de données variées (Superstore, Northwind, Stocks restauration, Retail Australie), en mettant en œuvre des modèles de données en étoile, des mesures DAX, des visualisations avancées (cartes, graphiques en aires, bulles animées, treemaps) et des filtres dynamiques.

Ces travaux confirment que la chaîne complète ETL (SSIS) — Entrepôt de données — Visualisation (Power BI) constitue le socle d'une architecture de Business Intelligence moderne, permettant aux décideurs de disposer d'informations fiables, actualisées et visuellement exploitables pour une prise de décision rapide et éclairée.